



UNIVERSITY OF
EASTERN FINLAND

Älykäs kirjanpito pienyrittäjille – tekoälypohjaisen yhdenkertaisen
kirjanpitosovelluksen suunnittelu, toteutus ja arviointi

Nuutti Nyyssönen
Tietojenkäsittelytieteiden
maisteriohjelma
Itä-Suomen yliopisto
24.7.2026

Tiivistelmä

Tämä tutkimus käsittelee tekoälypohjaisen kirjanpitosovelluksen suunnittelua, toteutusta ja arviointia suomalaisille pienyrittäjille. Tutkimuksen taustalla on pienten yritysten kirjanpidon aikaa vievä ja virhealtis luonne: moni yrittäjä hallinnoi kuittejaan epäorganisoidun Excel- taulukoiden avulla tai ulkoistaa kirjanpidon tilitoimistolle, mistä koituu jopa satoja euroja kuukausittaisia kustannuksia.

Tutkimusmenetelmänä käytettiin Design Science Research -viitekehystä, joka ohjasi työtä systemaattisesti ongelman tunnistamisesta toimivan ohjelmistoartefaktin suunnitteluun, toteutukseen ja arviointiin. Ohjelmiston kehitysmenetelmänä sovellettiin Agile-mallia iteratiivisine sprintteineen.

Toteutettu sovellus automatisoi kuittien käsittelyn yhdistämällä Google Vision API:n OCR-tekstintunnistuksen ja OpenAI GPT-4o-mini -kielimallin. Käyttäjä lataa kuitin sovellukseen, jonka jälkeen järjestelmä tunnistaa myyjän nimen, päivämäärän, kokonaissumman sekä ALV-erittelyn automaattisesti ja tallentaa tiedot tietokantaan. Sovellus tarjoaa lisäksi ALV-raportointiominaisuuden, AI-assistentin sekä tilausmallin Stripe-maksujärjestelmällä. Tekninen toteutus perustuu Next.js-, Express- ja PostgreSQL-pinoon, ja tietoturvassa noudatetaan OWASP Top 10 -suosituksia sekä GDPR-vaatimuksia.

Tarkkuuden arviointia varten kehitettiin automaattinen testausmenetelmä, jossa kuittien oikeat tiedot tallennettiin JSON-muotoiseen tiedostoon ja tuloksia verrattiin tietokannasta haettuihin arvoihin. Painettujen kuittien testissä 100 kuitin analysointi kesti 7 minuuttia ja 3 sekuntia ja tarkkuudeksi mitattiin 99,9 %. Käsillä kirjoitetuilla kuiteilla tarkkuus oli 96,8 %. Vastaava työmäärä manuaalisesti kirjattuna veisi kirjanpitäjältä arviolta 30–100 minuuttia.

Tulosten perusteella tekoäly soveltuu erinomaisesti kuittien automaattiseen käsittelyyn ja tuo merkittävää ajansäästöä pienyrittäjille. Tutkimus osoittaa, että prototyyppitasolla oleva sovellus vastaa asetettuihin tutkimuskysymyksiin: se nopeuttaa kirjanpitoa huomattavasti, saavuttaa

korkean tunnistustarkkuuden ja kohdistaa automaation juuri niihin työvaiheisiin, jotka kuormittavat pienyrittäjiä eniten.

1	Johdanto.....	6
	Tässä luvussa esitellään tutkimuksen tausta ja motivaatio, määritellään tutkimustavoitteet ja -kysymykset sekä kuvataan työn rakenne. Luku antaa lukijalle kokonaiskuvan siitä, miksi tämä tutkimus on tarpeellinen, mitä sillä pyritään saavuttamaan ja miten työ etenee.....	6
1.1	Tutkimuksen tausta ja motivaatio.....	6
1.2	Tutkimustavoitteet.....	6
1.3	Tutkimuskysymykset	7
1.4	Työn rakenne.....	7
2	Tutkimusmenetelmät	8
2.1	Design Science Research (DSR) -viitekehys.....	8
2.2	Menetelmät DSR-syklin erivaiheissa	10
2.3	Kehitysmenetelmä osana DSR-sykliä.....	11
3	Taustaa ja ongelma-analyysi (DSR vaihe 1).....	12
3.1	Pienten yritysten kirjanpidon haasteet	12
3.2	Tekoälyn mahdollisuudet kirjanpidon automatisoinnissa.....	12
4	Vaatimusmäärittely (DSR vaihe 2).....	14
4.1	Toiminnalliset vaatimukset	14
4.1.1	Käyttäjähallinta.....	14
4.1.2	Tositteiden hallinta	15
4.1.3	ALV-käsittely	15
4.1.4	Raportointi	16
4.1.5	AI-assistentti	16
4.1.6	Tilaukset ja maksaminen	17

4.2	Ei-toiminnalliset vaatimukset	18
4.3	Käyttötapaukset	19
4.3.1	Käyttäjähallinta.....	19
4.3.2	Tositteiden hallinta	21
4.3.3	ALV-käsittely	23
4.3.4	Raportointi	24
4.3.5	AI-assistentti	25
4.3.6	Tilaukset ja maksaminen	26
5	Järjestelmäsuunnittelu (DSR vaihe 3)	28
5.1	Arkkitehtuurimalli	28
5.2	Kerrosarkkitehtuuri ja teknologiapino	29
5.3	Arkkitehtuurikaavio.....	30
5.4	Tietoturva-arkkitehtuuri lyhyesti.....	30
5.5	AI-moduulin suunnittelu	31
5.6	Tietokantasuunnittelu	32
6	Testaussuunnitelma (DSR vaihe 3)	35
6.1	Testausstrategia	35
7	Toteutus (DSR vaihe 3)	37
7.1	Teknologiavalinnat ja perustelu	37
7.2	AI-mallin valinta	38
7.3	Backend-toteutus.....	39
7.4	Frontend-toteutus.....	41
7.5	Kehityshaasteet ja ratkaisut.....	43
7.6	Testaus	44
7.7	GDPR.....	45
7.8	OWASP Top 10, Audit tulokset sekä haavoittuvuudet.....	46

8	Demonstrointi ja arviointi (DSR vaiheet 4 ja 5).....	49
8.1	Käyttäytyvyys.....	51
9	Pohdinta.....	52
10	Lähteet	54

1 Johdanto

Tässä luvussa esitellään tutkimuksen tausta ja motivaatio, määritellään tutkimustavoitteet ja kysymykset sekä kuvataan työn rakenne. Luku antaa lukijalle kokonaiskuvan siitä, miksi tämä tutkimus on tarpeellinen, mitä sillä pyritään saavuttamaan ja miten työ etenee.

1.1 Tutkimuksen tausta ja motivaatio

Pienten yritysten kirjanpito on aikaa vievä prosessi etenkin yksittäiselle yrittäjälle. Moni pienenyritys käyttää kirjanpidossaan epäorganisioituja excel-tilioikeuksia tuloista ja menoista sekä säilyttävät fyysisesti valtavat määrät kuitteja näihin liittyen. Suomessa on yli 400 000 pien/mikroryitystä, jotka kattavat jopa 99 % Suomen yrityksistä (Suomen yrittäjät, 2026). Näistä suuriosa ulkoistaa oman yrityksen kirjanpidon tilitoimistolle ja maksaa siitä jopa useita satoja euroja kuukaudessa.

1.2 Tutkimustavoitteet

Tämän tutkimuksen tavoitteena on suunnitella ja toteuttaa tekoälypohjainen kirjanpitosovellus, joka automatisoi pienten yritysten kirjanpidon keskeisiä työvaiheita ja vähentää manuaalisen työn tarvetta. Muita alatavoitteita ovat esimerkiksi:

1. Tunnistaa pienten yritysten kipupisteet kirjanpidossa
2. Selvittää miten, AI soveltuu kirjanpidon automatisointiin
3. Suunnitella ja toteuttaa toimiva prototyyppi sovelluksesta
4. Arvioida sovelluksen hyödyt: ajansäästö, tarkkuus ja käytettävyys
5. Varmistaa, että ratkaisu täyttää Suomen kirjanpitolain ja GDPR:n vaatimukset

1.3 Tutkimuskysymykset

1. Kuinka paljon suunniteltu ohjelmisto auttaa säästämään aikaa?
2. Kuinka tarkkaan suunniteltu ohjelmisto luokittelee kuitit ja dokumentit?
3. Mitkä ovat pienten yritysten keskeisimmät haasteet kirjanpidossa, ja mihin työvaiheisiin automaatio toisi eniten hyötyä?

1.4 Työn rakenne

Työn tavoitteena on suunnitella ensin kattava dokumentaatio ongelmasta ja perehtyä perusteellisesti aiheeseen. Ennen itse devausta ja dokumentaation kirjoittamisen ohella, pyrin aloittamaan ohjelmiston suunnittelun (arkkitehtuurin suunnittelu: tietokanta, backend, UI/UX) lukuisilla ulkoisilla ohjelmistoilla. Tavoitteena on myös perehtyä lainsäädäntöön ja GDPR vaatimukseen tekoälyä käyttäessä kirjanpidossa.

Ohjelmisto toteutetaan Agile-kehitysmallilla, jossa järjestelmä kehitetään iteratiivisesti sprinteissä. Jokainen sprintti tuottaa toimivan osakokonaisuuden. Tämä mahdollistaa jatkuvan arvioinnin ja kehityksen ohjaamisen tulosten perusteella koko tutkimusprosessin ajan (Pasuksmit, 2021).

2 Tutkimusmenetelmät

Tässä osiossa perustellaan, miksi tietyt tutkimusstrategiat ja lähestymistavat on valittu tähän tutkimukseen sekä miten ne tukevat asetettujen tutkimuskysymysten ratkaisemista.

2.1 Design Science Research (DSR) -viitekehys

Tutkimuksen strategiana käytetään Design Science Research (DSR) -viitekehystä, joka on vakiintunut lähestymistapa erityisesti tietojärjestelmätieteessä. DSR soveltuu tähän tutkimukseen erityisen hyvin, koska tieteellinen kontribuutio ei synny pelkästään ilmiön kuvaamisesta tai selittämisestä, vaan konkreettisen artefaktin eli tässä tapauksessa AI-pohjaisen kirjanpitosovelluksen suunnittelusta, toteutuksesta ja arvioinnista.

DSR-prosessin vaiheet (Peppers, 2007)

1. Ongelman tunnistaminen ja motivointi.

Määritellään tutkimusongelma ja perustellaan ratkaisun arvo. Ongelma pyritään pilkkomaan käsitteellisesti, jotta artefakti voi kattaa sen monimutkaisuuden.

2. Ratkaisun tavoitteiden määrittely.

Johdetaan ratkaisun tavoitteet ongelmamäärittelystä ja tietämyksestä siitä, mikä on mahdollista. Tavoitteet voivat olla kvantitatiivisia tai kvalitatiivisia.

3. Suunnittelu ja kehittäminen.

Luodaan artefakti eli malli, menetelmä tai instanssi. Määritellään artefaktin haluttu toiminnallisuus ja arkkitehtuuri, jonka jälkeen artefakti rakennetaan.

4. Demonstrointi

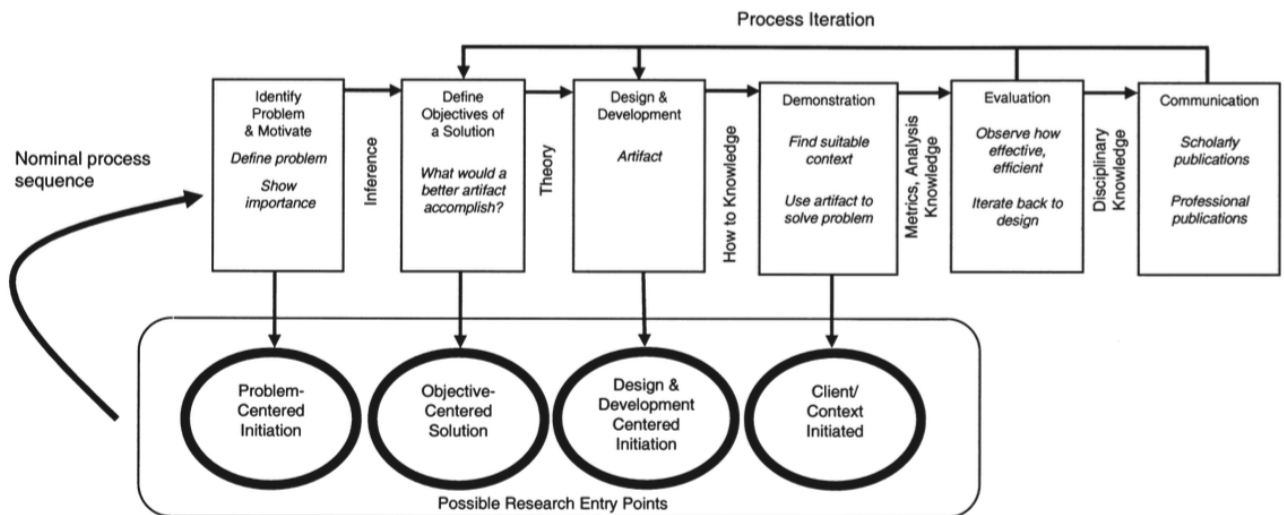
Osoitetaan artefaktin kyky ratkaista ongelma esimerkiksi kokeilun, simulaation tai tapaustutkimuksen avulla.

5. Arviointi

Mitataan ja arvioidaan, kuinka hyvin artefakti tukee ongelman ratkaisua vertaamalla asetettuja tavoitteita todellisiin tuloksiin. Tarvittaessa palataan vaiheeseen 3 artefaktin parantamiseksi.

6. Kommunikointi

Viestitään ongelma, artefakti, sen hyödyllisyys ja uutuusarvo sekä suunnittelun tiukkuus tutkijayhteisölle ja muille relevanteille sidosryhmille.



(Peffers, 2007)

2.2 Menetelmät DSR-syklin erivaiheissa

Koska DSR ei itsessään määrää käytettäviä aineistonkeruu- tai analyysimenetelmiä, tutkimuksessa yhdistetään kvalitatiivista ja kvantitatiivista lähestymistapaa prosessin eri vaiheissa. Kvalitatiivisen lähestymistavan tavoitteena on ymmärtää ja selittää ilmiötä syvällisesti, kun taas kvantitatiivisen lähestymistavan tavoitteena on mitata ja yleistää tuloksia (Abbadia, 2023).

Vaihe 1 – Ongelman tunnistaminen ja motivointi (kvalitatiivinen):

- Suomalaisten pienyrittäjien manuaalisen kirjanpityön ja kuittien kasaantumisen kartoitus laadullisesti

Vaihe 2 – Ratkaisun tavoitteiden määrittely (kvalitatiivinen):

- Käyttäjätarpeiden pohjalta johdettu vaatimusmäärittely

Vaihe 3 – Suunnittelu ja kehittäminen (kvalitatiivinen perustelu):

- Arkkitehtuuristen ja teknologisten valintojen perustelu

Vaihe 4 – Demonstrointi (kuvaileva):

- Sovelluksen toiminnan osoittaminen esimerkkitapausten tai -datan avulla

Vaihe 5 – Arviointi (kvantitatiivinen):

- OCR-tunnistuksen tarkkuus prosentteina
- Ajansäästö manuaaliseen kirjanpitoon verrattuna

Vaihe 6 – Kommunikointi:

- Tulosten raportointi tässä maisteriopintojen erikoistyössä

2.3 Kehitysmenetelmä osana DSR-sykliä

Ohjelmiston kehitysmenetelmänä käytetään Agile-mallia, joka tukee suoraan DSR-prosessin vaiheiden 3–5 (suunnittelu ja kehittäminen, demonstrointi, arviointi) iteratiivista luonnetta.

Peffer ym. (2007) toteavat, että arviointivaiheesta (vaihe 5) voidaan tarvittaessa palata takaisin suunnittelu- ja kehitysvaiheeseen (vaihe 3) artefaktin parantamiseksi – tämä vastaa suoraan Agile-mallin sprintteihin perustuvaa toimintatapaa.

Kehitystyö jakautuu kahden viikon sprintteihin, joista jokainen tuottaa arvioitavissa olevan osakokonaisuuden. Agile valittiin, koska:

- vaatimukset voivat tarkentua tutkimuksen edetessä
- jokainen sprintti mahdollistaa varhaisen palautteen keräämisen ja suunnan korjaamisen
- iteratiivinen kehitys vastaa suoraan DSR-prosessin logiikkaa, jossa artefaktia rakennetaan (vaihe 3), demonstroidaan (vaihe 4) ja arvioidaan (vaihe 5) toistuvasti, kunnes se täyttää asetetut tavoitteet.

3 Taustaa ja ongelma-analyysi (DSR vaihe 1)

3.1 Pienten yritysten kirjanpidon haasteet

Pienten yritysten kirjanpidon haasteet liittyvät useimmiten ajankäyttöön sekä resurssien puutteeseen. Ajanhallinta ja rutiinit ovat oleellinen osa etenkin yksittäisten yrittäjien arkea. Kuukausittaiset kuitit ja tositteet saattavat kasaantua, jolloin taloustilanne ei ole reaaliajassa yrityksen johdon tiedossa. Myös kirjanpito nähdään usein vain pakollisena "verottajaa varten tehtävänä pakkopullana", eikä sen tuomaa dataa osata tai ehditä hyödyntää liiketoiminnan kehittämisessä.

Ulkoistettu kirjanpito maksaa tyypillisesti noin 50–250 euroa kuukaudessa yrityksen koon mukaan, mutta oman taloushenkilöstön palkkaaminen on pienelle yritykselle yleensä liian kallista (KT Counter Oy, 2026).

Kirjanpito saattaa sisältää myös paljon teknisiä haasteita. Suuri osa pienyrityksistä käyttää kirjanpidossaan yksinkertaisia Excel-taulukoita tai pahimmassa tapauksessa pelkkiä paperisia kirjanpitolihkoja. Vaikka markkinoilla on ohjelmistoratkaisuja kuten Procountor, Netvisor ja Visma, ne ovat usein suunniteltu isompien organisaatioiden tarpeisiin ja ovat käyttöliittymiltään monimutkaisia sekä hinnoittelultaan kalliita pienimmille toimijoille. Lisäksi nykyiset ratkaisut vaativat edelleen paljon manuaalista tietojen syöttämistä eivätkä hyödynnä tekoälyn mahdollisuuksia automatisoinnissa merkittävässä määrin.

3.2 Tekoälyn mahdollisuudet kirjanpidon automatisoinnissa

Tekoälyllä on paljon mahdollisuuksia kirjanpidon automatisoinnissa. Yksi kirjanpidon eniten aikaa vievistä työvaiheista on tositteiden ja kuittien manuaalinen käsittely. Optical Character Recognition (OCR) -teknologia mahdollistaa kuittien ja laskujen automaattisen digitalisoinnin ja tietojen poiminnan. Tekoälypohjainen OCR tunnistaa kuitista automaattisesti esimerkiksi:

- Myyjän nimen ja Y-tunnuksen
- Päivämäärän ja summan

- ALV-prosentin ja ALV-määrän
- Maksutavan

Modernit OCR-ratkaisut, kuten Google Vision API ja AWS Textract, saavuttavat yli 95 % tarkkuuden selkeistä kuiteista, mikä eliminoi käytännössä manuaalisen tietojen syöttämisen tarpeen. (Dilmegani, 2026)

4 Vaatimusmäärittely (DSR vaihe 2)

Tämän luvun tarkoituksena on kuvata lukijalle ohjelmiston toiminnalliset vaatimukset, ei-toiminnalliset vaatimukset sekä käyttötapaukset. Luvun luettuaan lukija ymmärtää, mitä toimintoja sovelluksella on, kuinka toiminnot toimivat suorituskyvyltään sekä minkälaisen käyttäjäpolun erilaiset toiminnot vaativat.

4.1 Toiminnalliset vaatimukset

Alla on kuvattuna taulukot toiminnallisista vaatimuksista. Jokainen toiminta on luokiteltu ID:llä (FR = Functional Requirement), jotta eri toiminnot on helppo erotella toisistaan sekä näihin ID-arvoihin voidaan viitata kätevästi dokumentaation testaus osiossa.

4.1.1 Käyttäjähallinta

ID	Vaatus	Prioriteetti	Hyvaksymiskriteeri
FR-001	Uuden käyttäjän luominen	Pakollinen	- Järjestelmä estää rekisteröinnin, jos sähköposti on jo käytössä ja näyttää virheilmoituksen - Salasana hylätään, jos se ei täytä vaatimuksia (esim. min. 8 merkkiä) - Onnistuneen rekisteröinnin jälkeen käyttäjä ohjataan kirjautumissivulle
FR-002	Olemassa olevaan käyttäjään kirjautuminen	Pakollinen	- Oikeilla tunnuksilla käyttäjä pääsee sisään ja ohjataan etusivulle - Väärillä tunnuksilla näytetään virheilmoitus
FR-003	Salasan	Pakollinen	- Käyttäjä pääsee salasanan palautussivulle, missä syötetään sähköpostiosoite - Käyttäjän sähköpostiin tulee linkki salasanan palautusta varten - Jos sähköpostia ei löydy järjestelmästä, näytetään neutraali ilmoitus

4.1.2 Tositteiden hallinta

ID	Vaatus	Prioriteetti	Hyväksymiskriteeri
FR-004	Käyttäjä voi ladata kuvan tai PDF-tiedoston kuitista järjestelmään	Pakollinen	- Käyttäjällä on kaksi vaihtoehtoa ladata kuitti järjestelmään, tiedoston lataaminen tai kuvan ottaminen - Kuvan ottaminen pyytää oikeuksia kameraan - PDF-tiedoston voi drag and drop -tyyliin tai hakemalla laitteesta
FR-005	Järjestelmä tunnistaa automaattisesti kuitin summan, päivämäärän ja myyjän OCR-teknologialla	Pakollinen	- Käyttäjä näkee AI:n yhteenvedon kuitista - Käyttäjä joko hyväksyy, hylkää tai muokkaa itse AI:n antamia tuloksia
FR-006	Käyttäjä voi muokata OCR:n tunnistamia tietoja manuaalisesti	Pakollinen	Käyttäjä pääsee itse muokkaamaan AI:n antamia yksittäisiä tuloksia sekä korjaamaan mahdollisia virheitä
FR-007	Järjestelmä tallentaa alkuperäisen kuittikuvan arkistoon	Pakollinen	- Käyttäjä pääsee tarkastelemaan kaikki tallentamia dokumentteja - Dokumentit on lajiteltu päivämäärän mukaan - Dokumentteja pystyy poistamaan järjestelmästä

4.1.3 ALV-käsittely

ID	Vaatus	Prioriteetti	Hyväksymiskriteeri
FR-008	Järjestelmä tunnistaa ja laskee ALV:n automaattisesti kuitista	Pakollinen	- AI-yhteenvedon yhteydessä käyttäjä näkee ALV:n - Käyttäjä voi muokata ALV:ia
FR-009	Järjestelmä generoi ALV-yhteenvedon valitulle aikavälille	Pakollinen	- Käyttäjä voi valita haluamansa aikavälin päivämäärävalitsimella - Järjestelmä laskee yhteenveoton vain valitulle aikavälille osuvat tapahtumat - Yhteenveoto erittelee myynnin ALV:n ja ostojen ALV:n erikseen - Yhteenveoto erittelee tapahtumat ALV-prosentin mukaan (0 %, 10 %, 14 %, 24 %)

4.1.4 Raportointi

ID	Vaatus	Prioriteetti	Hyväksymiskriteeri
FR-010	Käyttäjä voi tarkastella tuloja ja menoja kuukausi- ja vuositasolla	Pakollinen	- Käyttäjä voi valita haluamansa aikavälin tarkastelulle - Järjestelmä laskee automaattisesti menot ja tulot sekä näyttää ne käyttäjälle intuitiivisesti - Menoista ja tuloista voi ladata PDF-raportin
FR-011	Järjestelmä generoi tuloslaskelman automaattisesti	Pakollinen	- Käyttäjä voi valita tuloslaskelman ajanjakson kuukausi-, kvartaali- tai vuositasolla - Järjestelmä generoi tuloslaskelman automaattisesti ilman käyttäjän manuaalista syöttöä - Tuloslaskelma sisältää vertailutiedot edelliseen vastaavaan ajanjaksoon - Järjestelmä päivittää tuloslaskelman automaattisesti, kun uusia tapahtumia lisätään - Tuloslaskelma on exportoitavissa PDF- ja Excel-muotoon
FR-012	Käyttäjä voi viedä kirjanpilotdata Excel- tai PDF-muotoon	Pakollinen	- Käyttäjälle näytetään painike, mistä voi ladata datan valitsemaan muotoon - Käyttäjä voi valita aikavälin, mistä haluaa ladata datan

4.1.5 AI-assistentti

ID	Vaatus	Prioriteetti	Hyväksymiskriteeri
FR-013	Käyttäjä voi esittää kirjanpitoon liittyviä kysymyksiä luonnollisella kielellä	Pakollinen	- Jokaisella sivulla näytetään chatti kuvake, jota käyttäjä voi klikata aukaistakseen chatin AI-assistentin kanssa - Chattiin voi kirjoittaa millä kielellä tahansa - AI-assistentti voi käydä läpi dokumentteja, mikäli sillä on oikeus

4.1.6 Tilaukset ja maksaminen

ID	Vaatusus	Prioriteetti	Hyväksymiskriteeri
FR-014	Järjestelmä tarjoaa maksuttoman demon valmiilla datalla	Pakollinen	- Käyttäjä pystyy testaamaan demoa ilman käyttäjää - Demossa on valmis data toimintoja varten - Demoon ei voi syöttää omaa dataa testattavaksi
FR-015	Käyttäjä voi ostaa palveluun kuukausitilauksen	Pakollinen	- Käyttäjälle näytetään selkeästi tilauksen hinta ennen ostamista - Käyttäjä voi syöttää luottokorttitiedot turvallisesti maksulomakkeella - Käyttäjä voi tarkastella tilauksen tietoja ja laskuhistoriaa omalta tilisivulta - Onnistuneen maksun jälkeen käyttäjän tilaus aktivoituu välittömästi - Onnistuneen maksun jälkeen käyttäjä saa vahvistussähköpostin tilauksesta
FR-016	Käyttäjä voi valita kuukausi- tai vuositilauksen välillä	Vapaaehtoinen	- Käyttäjälle näytetään tilausvaihtoehtoja - Vuositilauksen hinta on alhaisempi suhteessa kuukausitilaukseen
FR-017	Järjestelmä integroi maksunvälittäjän kuten Stripe tai PayPal maksujen käsittelyyn	Pakollinen	- Järjestelmä käyttää Stripen tai PayPalin virallista API:a maksujen käsittelyyn - Integraatio tukee yleisimpiä luottokortteja kuten Visa, Mastercard ja American Express - Järjestelmä päivittää käyttäjän tilauksen tilan automaattisesti webhook-ilmoituksen perusteella - Järjestelmä tukee tilauksen peruutusta maksunvälittäjän kautta automaattisesti

4.2 Ei-toiminnalliset vaatimukset

Alla on kuvattuna taulukko ei-toiminnallisista vaatimuksista. Jokainen toiminta on luokiteltu ID:llä (NFR = Non-Functional Requirement), jotta eri toiminnot on helppo erotella toisistaan sekä näihin ID-arvoihin voidaan viitata kätevästi dokumentaation testaus osiossa.

ID	Vaatus	Prioriteetti	Hyvaksymiskriteeri
NFR-001	Sovelluksen latausajan tulee olla alle 3 sekuntia	Pakollinen	- Etusivu latautuu alle 3 sekunnissa normaalilla internetyhteydellä - Tuloslaskelma ja ALV-yhteenveto generoituvat alle 5 sekunnissa
NFR-002	OCR-tunnistuksen tarkkuus selkeistä kuiteista	Pakollinen	Järjestelmä tunnistaa summan, päivämäärän ja myyjän vähintään 95 % tarkkuudella selkeistä kuiteista
NFR-003	Sovelluksen tulee olla saatavilla 99,9 % ajasta	Pakollinen	- Suunnittelematon käyttökatko ei ylitä 8 tuntia vuodessa - Käyttäjälle näytetään ilmoitus suunnitelluista huoltokatkoksista etukäteen
NFR-004	Käyttäjän data on suojattava GDPR:n mukaisesti	Pakollinen	- Käyttäjän data tallennetaan EU:n alueelle - Käyttäjä voi pyytää oman datansa poistamista - Tietosuojaseloste on nähtävillä sovelluksessa
NFR-005	Kaikki tietoliikenne on salattava	Pakollinen	- Sovellus käyttää HTTPS-yhteyttä - Luottokorttitietoja ei tallenneta omalle palvelimelle - Salasanat tallennetaan tietokantaan hashattuna
NFR-006	Sovelluksen tulee toimia yleisimmillä selaimilla	Pakollinen	- Sovellus toimii oikein Chrome, Firefox, Safari ja Edge selaimilla - Sovellus toimii mobiililaitteilla iOS ja Android alustoilla
NFR-007	Sovelluksen käyttöliittymän tulee olla intuitiivinen	Pakollinen	Uusi käyttäjä pystyy lisäämään uuden järjestelmään ilman ohjeita alle 2 minuutissa
NFR-008	Järjestelmän tulee skaalautua käyttäjämäärän kasvaessa	Pakollinen	Järjestelmä toimii oikein vähintään 1000 samanaikaisella käyttäjällä

ID	Vaatus	Prioriteetti	Hyväksymiskriteeri
			Suorituskyky ei heikkene merkittävästi kuormituksen kasvaessa
NFR-009	Sovelluksen tulee noudattaa Suomen kirjanpitolakia	Pakollinen	- Tuloslaskelma noudattaa kirjanpitolain mukaista rakennetta - ALV-käsittely noudattaa Verohallinnon ohjeistusta

4.3 Käyttötapaukset

Tässä osiossa kuvataan järjestelmän keskeisimmät käyttötapaukset. Jokainen käyttötapaus kuvaa yksityiskohtaisesti, miten aktori vuorovaikuttaa järjestelmän kanssa tietyn tavoitteen saavuttamiseksi.

4.3.1 Käyttäjähallinta

UC-001 – Rekisteröityminen järjestelmään

Kenttä	Kuvaus
Tunnus	UC-001
Nimi	Rekisteröityminen järjestelmään
Aktori	Uusi käyttäjä
Kuvaus	Käyttäjä luo uuden tilin sähköpostilla ja salasanalla.
Esiehdot	- Käyttäjällä ei ole olemassa olevaa tiliä - Rekisteröintisivu on auki

Kenttä	Kuvaus
Päävirtaus	1. Käyttäjä avaa rekisteröintisivun 2. Käyttäjä syöttää sähköpostiosoitteen ja salasanan 3. Käyttäjä hyväksyy käyttöehdot 4. Käyttäjä klikkaa Rekisteröidy-painiketta 5. Järjestelmä luo uuden tilin ja ohjaa käyttäjän kirjautumissivulle
Vaihtoehtoinen virtaus	6. Sähköposti on jo käytössä: järjestelmä näyttää virheilmoituksen 7. Salasana ei täytä vaatimuksia: järjestelmä näyttää ohjeen vaatimuksista
Jälkiehdot	• Uusi käyttäjätili on luotu järjestelmään • Käyttäjä ohjataan kirjautumissivulle
Liittyvät vaatimukset	FR-001

UC-002 – Kirjautuminen järjestelmään

Kenttä	Kuvaus
Tunnus	UC-002
Nimi	Kirjautuminen järjestelmään
Aktori	Rekisteröitynyt käyttäjä
Kuvaus	Käyttäjä kirjautuu järjestelmään olemassa olevilla tunnuksilla.
Esiehdot	• Käyttäjällä on olemassa oleva tili • Kirjautumissivu on auki
Päävirtaus	8. Käyttäjä avaa kirjautumissivun 9. Käyttäjä syöttää sähköpostiosoitteen ja salasanan 10. Käyttäjä klikkaa Kirjaudu-painiketta 11. Järjestelmä tarkistaa tunnukset 12. Järjestelmä ohjaa käyttäjän etusivulle
Vaihtoehtoinen virtaus	13. Väärät tunnukset: järjestelmä näyttää virheilmoituksen 14. Käyttäjä unohtanut salasanan: käyttäjä siirtyy salasanan palautukseen (UC-003)
Jälkiehdot	• Käyttäjä on kirjautunut sisään • Käyttäjä näkee etusivun

Kenttä	Kuvaus
Liittyvät vaatimukset	FR-002

UC-003 – Salasanan palautus

Kenttä	Kuvaus
Tunnus	UC-003
Nimi	Salasanan palautus
Aktori	Rekisteröitynyt käyttäjä
Kuvaus	Käyttäjä palauttaa unohdetun salasanan sähköpostilinkin kautta.
Esiehdot	- Käyttäjällä on olemassa oleva tili - Käyttäjä ei muista salasanaansa
Päävirtaus	- Käyttäjä klikkaa Unohditko salasanasi? -linkkiä - Käyttäjä syöttää sähköpostiosoitteensa - Järjestelmä lähettää palautuslinkin sähköpostiin - Käyttäjä avaa linkin ja syöttää uuden salasan - Järjestelmä päivittää salasan ja ohjaa kirjautumissivulle
Vaihtoehtoinen virtaus	Sähköpostia ei löydy: järjestelmä näyttää neutraalin ilmoituksen tietoturvan vuoksi
Jälkiehdot	- Salasana on vaihdettu - Käyttäjä ohjataan kirjautumissivulle
Liittyvät vaatimukset	FR-003

4.3.2 Tositteiden hallinta

UC-004 – Kuitin lataaminen järjestelmään

Kenttä	Kuvaus
Tunnus	UC-004

Kenttä	Kuvaus
Nimi	Kuitin lataaminen järjestelmään
Aktori	Kirjautunut käyttäjä
Kuvaus	Käyttäjä lataa kuitin järjestelmään joko ottamalla kuvan tai lataamalla tiedoston.
Esiehdot	- Käyttäjä on kirjautunut sisään - Käyttäjällä on kuitti tallennettavaksi
Päävirtaus	21. Käyttäjä siirtyy Lisää kuitti -sivulle 22. Käyttäjä valitsee lataustavan: kamera tai tiedosto 23. Käyttäjä ottaa kuvan tai valitsee tiedoston laitteelta 24. Järjestelmä lataa tiedoston ja näyttää esikatselun 25. Käyttäjä vahvistaa lähetyksen
Vaihtoehtoinen virtaus	26. Tiedostomuoto ei ole tuettu: järjestelmä näyttää virheilmoituksen 27. Tiedosto on liian suuri: järjestelmä pyytää pienempää tiedostoa
Jälkiehdot	- Kuitti on tallennettu järjestelmään - OCR-analyysi käynnistyy automaattisesti
Liittyvät vaatimukset	FR-004

UC-005 – OCR-tunnistuksen tulosten tarkistaminen

Kenttä	Kuvaus
Tunnus	UC-005
Nimi	OCR-tunnistuksen tulosten tarkistaminen
Aktori	Kirjautunut käyttäjä
Kuvaus	Käyttäjä tarkistaa ja hyväksyy tai muokkaa OCR:n tunnistamat tiedot kuitista.

Kenttä	Kuvaus
Esiehdot	- Kuitti on ladattu järjestelmään - OCR-analyysi on valmis
Päävirtaus	28. Järjestelmä näyttää OCR:n tunnistaman yhteenvedon kuitista 29. Käyttäjä tarkistaa tunnistetut tiedot: summa, päivämäärä, myyjä ja ALV 30. Käyttäjä hyväksyy tiedot klikkaamalla Hyväksy-painiketta 31. Järjestelmä tallentaa tiedot kirjanpitoon
Vaihtoehtoinen virtaus	32. Tunnistetut tiedot ovat virheellisiä: käyttäjä muokkaa tietoja manuaalisesti ennen hyväksymistä 33. Käyttäjä hylkää kuitin: järjestelmä poistaa kuitin tai pyytää lataamaan uudelleen
Jälkiehdot	- Kuitin tiedot on tallennettu kirjanpitoon - Kuitti näkyy arkistossa
Liittyvät vaatimukset	FR-005, FR-006

4.3.3 ALV-käsittely

UC-006 – ALV-yhteenvedon generointi

Kenttä	Kuvaus
Tunnus	UC-006
Nimi	ALV-yhteenvedon generointi
Aktori	Kirjautunut käyttäjä
Kuvaus	Käyttäjä generoi ALV-yhteenvedon haluamaltaan aikaväliltä ALV-ilmoitusta varten.
Esiehdot	- Käyttäjä on kirjautunut sisään - Järjestelmässä on kirjauksia valitulle aikavälille

Kenttä	Kuvaus
Päävirtaus	34. Käyttäjä siirtyy ALV-yhteenveto-sivulle 35. Käyttäjä valitsee aikavälin päivämäärävalitsimella 36. Käyttäjä klikkaa Generoi yhteenveto -painiketta 37. Järjestelmä laskee ja näyttää ALV-yhteenvedon eriteltynä ALV-prosenteittain 38. Käyttäjä voi ladata yhteenvedon PDF-muotoon
Vaihtoehtoinen virtaus	39. Valitulle aikavälille ei löydy tapahtumia: järjestelmä näyttää ilmoituksen
Jälkiehdot	- ALV-yhteenveto on generoitu - Käyttäjä voi hyödyntää sitä ALV-ilmoituksen tekemiseen
Liittyvät vaatimukset	FR-009

4.3.4 Raportointi

UC-007 – Tuloslaskelman generointi

Kenttä	Kuvaus
Tunnus	UC-007
Nimi	Tuloslaskelman generointi
Aktori	Kirjautunut käyttäjä
Kuvaus	Käyttäjä generoi tuloslaskelman haluamaltaan ajanjaksolta.
Esiehdot	- Käyttäjä on kirjautunut sisään - Järjestelmässä on kirjauksia
Päävirtaus	40. Käyttäjä siirtyy Raportit-sivulle 41. Käyttäjä valitsee ajanjakson: kuukausi, kvartaali tai vuosi 42. Järjestelmä generoi tuloslaskelman automaattisesti 43. Käyttäjä tarkastelee tuloslaskelmaa vertailutietoineen 44. Käyttäjä lataa tuloslaskelman PDF- tai Excel-muodossa

Kenttä	Kuvaus
Vaihtoehtoinen virtaus	45. Tiedot puuttuvat: järjestelmä näyttää virheilmoituksen ja kertoo mistä tiedot puuttuvat
Jälkiehdot	Tuloslaskelma on generoitu ja ladattavissa
Liittyvät vaatimukset	FR-011, FR-012

4.3.5 AI-assistentti

UC-008 – Kysymyksen esittäminen AI-assistentille

Kenttä	Kuvaus
Tunnus	UC-008
Nimi	Kysymyksen esittäminen AI-assistentille
Aktori	Kirjautunut käyttäjä
Kuvaus	Käyttäjä esittää kirjanpitoon liittyvän kysymyksen AI-assistentille luonnollisella kielellä.
Esiehdot	Käyttäjä on kirjautunut sisään
Päävirtaus	46. Käyttäjä klikkaa chat-kuvaketta sivun reunassa 47. Chat-ikkuna avautuu 48. Käyttäjä kirjoittaa kysymyksensä 49. AI-assistentti analysoi kysymyksen ja vastaa 50. Käyttäjä voi jatkaa keskustelua tai sulkea chatin
Vaihtoehtoinen virtaus	51. Kysymys liittyy käyttäjän dokumentteihin: assistentti pyytää lupaa tarkastella dokumentteja ennen vastausta
Jälkiehdot	Käyttäjä on saanut vastauksen kysymykseensä
Liittyvät vaatimukset	FR-013

4.3.6 Tilaukset ja maksaminen

UC-009 – Kuukausitilauksen ostaminen

Kenttä	Kuvaus
Tunnus	UC-009
Nimi	Kuukausitilauksen ostaminen
Aktori	Rekisteröitynyt käyttäjä
Kuvaus	Käyttäjä ostaa kuukausitilauksen maksullisiin ominaisuuksiin.
Esiehdot	- Käyttäjä on kirjautunut sisään - Käyttäjällä ei ole voimassa olevaa tilausta
Päävirtaus	- Käyttäjä siirtyy Tilaus-sivulle - Käyttäjä valitsee kuukausitilauksen - Käyttäjä syöttää luottokorttitiedot turvalliseen maksulomakkeeseen - Käyttäjä klikkaa Osta-painiketta - Järjestelmä käsittelee maksun Strapen kautta - Järjestelmä aktivoi tilauksen välittömästi ja lähettää vahvistussähköpostin
Vaihtoehtoinen virtaus	Maksu epäonnistuu: järjestelmä näyttää virheilmoituksen ja pyytää tarkistamaan tiedot
Jälkiehdot	- Tilaus on aktivoitu - Käyttäjä saa vahvistussähköpostin - Käyttäjä pääsee kaikkiin ominaisuuksiin
Liittyvät vaatimukset	FR-015, FR-017

UC-010 – Demon käyttäminen

Kenttä	Kuvaus
Tunnus	UC-010
Nimi	Demon käyttäminen

Kenttä	Kuvaus
Aktori	Vierailija (ei kirjautunut)
Kuvaus	Vierailija kokeilee sovellusta valmiilla demotiedoilla ilman rekisteröitymistä.
Esiehdot	Käyttäjä ei ole kirjautunut sisään
Päävirtaus	59. Vierailija siirtyy etusivulle 60. Vierailija klikkaa Kokeile demoa -painiketta 61. Järjestelmä lataa demoympäristön valmiilla testidatalla 62. Vierailija voi selata toimintoja vapaasti
Vaihtoehtoinen virtaus	63. Vierailija yrittää syöttää omaa dataa: järjestelmä estää sen ja näyttää ohjeen rekisteröitymisestä
Jälkiehdot	- Vierailija on tutustunut sovelluksen toimintoihin - Järjestelmä ei tallenna mitään dataa
Liittyvät vaatimukset	FR-014

5 Järjestelmäsuunnittelu (DSR vaihe 3)

Tämän luvun tarkoitus on antaa lukijalle selkeä kuva järjestelmän arkkitehtuurista sekä teknologiapinosta mitä projektissa käytetään. Ideana on myös, että luvun jälkeen lukija myös ymmärtää valinnat ja perustelut näiden teknologiavalintojen taustalla.

5.1 Arkkitehtuurimalli

Järjestelmä rakennetaan noudattaen client-server-arkkitehtuurin periaatteita. Client-server on arkkitehtuurimalli, jossa sovellus jakautuu kahteen erilliseen osaan: clientiin eli käyttöliittymään sekä serveriin eli palvelimeen. Client vastaa käyttäjälle näkyvästä käyttöliittymästä ja lähettää pyyntöjä palvelimelle, joka puolestaan käsittelee pyynnöt, hallinnoi tietokantaa ja palauttaa vastauksen takaisin clientille. Tämä jako mahdollistaa selkeän vastuunjaon järjestelmän eri osien välillä sekä helpottaa sovelluksen ylläpitoa ja skaalautuvuutta (Hura, 1995). Ideana on siis, että Frontend puoli kommunikoi backendiin REST API:n kautta. REST on arkkitehtuuri tyyli, joka mahdollistaa yhtenäisen rajapinnan, jonka avulla kommunikointi on ennustettavaa ja johdonmukaista. Rajapinta myös toteutuu standardoitujen http-metodien, kuten GET, POST, PUT ja DELETE avulla (Gupta, 2025).

Järjestelmä on suunniteltu myös monoliittisen arkkitehtuurin periaattein. Monoliittinen arkkitehtuuri yhdistää sovelluksen kaikki komponentit: käyttöliittymän, sovelluslogiikan ja tietokantakerroksen yhdeksi yhtenäiseksi kokonaisuudeksi, joka kehitetään ja julkaistaan yhtenä yksikkönä. Tämä lähestymistapa yksinkertaistaa kehitystä ja käyttöönottoa, sillä kaikki osat sijaitsevat samassa koodipohjassa. Monoliittisen arkkitehtuurin keskeisiä etuja ovat yksinkertaisempi testaus ja debuggaus keskitetyn koodipohjan ansiosta sekä parempi suorituskyky, koska komponenttien välinen kommunikaatio tapahtuu saman prosessin sisällä ilman verkkoviivettä. Haasteina ovat kuitenkin rajoittunut skaalautuvuus, sillä sovelluksen laajentaminen vaatii usein koko järjestelmän kopioimisen yksittäisten osien sijaan, sekä joustamattomuus, koska pienikin muutos edellyttää koko sovelluksen uudelleenjulkaisua. Lisäksi koodipohjan monimutkaisuus kasvaa sovelluksen laajentuessa, mikä vaikeuttaa ylläpitoa (Kamisetty, 2023).

5.2 Kerrosarkkitehtuuri ja teknologiapino

Järjestelmä koostuu neljästä eri kerroksesta. Nämä kerrokset ovat: Frontend, Backend, Tietokanta ja AI-palvelut sekä muut ulkoiset palvelut. Frontend-kerros toteutetaan Next.js-sovelluskehityksellä, joka rakentuu React-kirjaston päälle. Next.js mahdollistaa sekä palvelinpuolen renderöinnin että staattisen sivugeneraation, mikä parantaa sovelluksen suorituskykyä ja hakukoneoptimointia. Frontend vastaa käyttäjälle näkyvästä käyttöliittymästä ja kommunikoi backend-kerroksen kanssa REST API -rajapinnan kautta (Next.js Documentation, 2024).

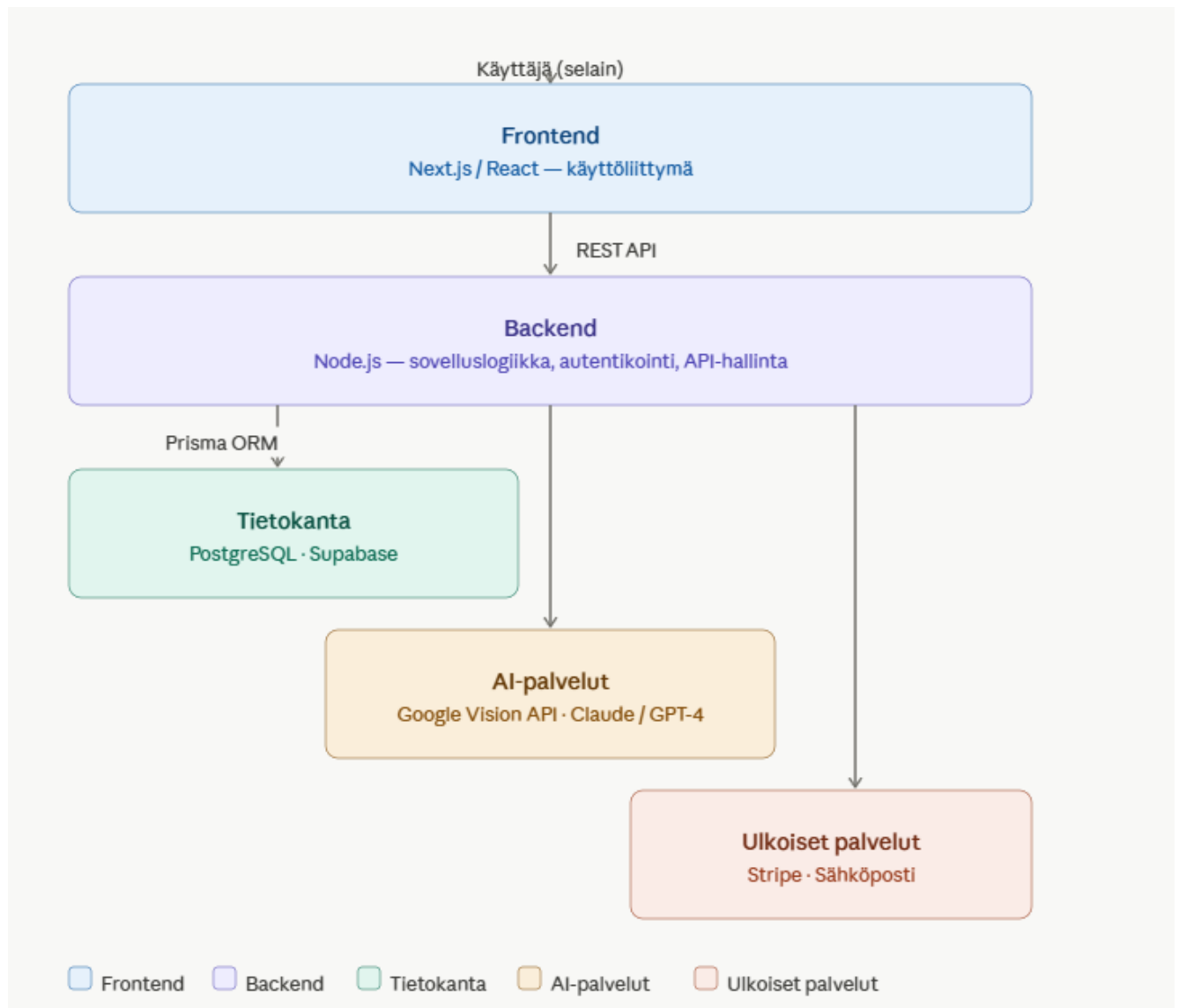
Backend-kerros toteutetaan Node.js-ajoympäristöllä, joka huolehtii sovelluksen liiketoimintalogiikasta, autentikoinnista sekä ulkoisten palveluiden integraatioista. Backend toimii välikerroksena frontendin, tietokannan ja AI-palveluiden välillä ja tarjoaa frontendille REST API -rajapinnan datan hakemiseen ja muokkaamiseen.

Tietokantakerros rakentuu PostgreSQL-relaatiotietokannalle, johon tallennetaan kaikki sovelluksen data, kuten käyttäjätiedot, kuitit ja tilaukset. Tietokantakerroksen ja backend-kerroksen väliseen kommunikaatioon käytetään Prisma ORM:ia, joka tarjoaa tyypitetyn ja selkeän rajapinnan tietokantakyselyille ilman manuaalisen SQL:n kirjoittamista. Prisma yksinkertaistaa tietokantaoperaatioita ja vähentää virhealttiutta kehitysvaiheessa. PostgreSQL valittiin sen luotettavuuden, ACID-yhteensopivuuden sekä monipuolisten kyselyominaisuuksien vuoksi.

AI- ja ulkoisten palveluiden kerros koostuu neljästä integraatiosta. OCR-tunnistus toteutetaan Google Vision API:n avulla, joka vastaa kuittien automaattisesta digitalisoinnista ja tietojen poiminnasta. Kirjanpitoassistentti rakennetaan hyödyntämällä suurta kielimallia, kuten Claude tai GPT-4, joka mahdollistaa luonnollisen kielen käsittelyn ja käyttäjän ohjaamisen kirjanpito prosessissa. Maksujen käsittely hoidetaan Stripe-integraation kautta, joka vastaa

tilausten ja maksujen hallinnasta. Dokumenttien tallennus puolestaan hoidetaan Supabasella, johon kuvat kuiteista tai PDF-tiedostot tallentuvat. Näistä dokumenteista saadaan kätevästi Supabasen tallennuspolku tallennettua PostgreSQL-tietokantaan.

5.3 Arkkitehtuurikaavio



5.4 Tietoturva-arkkitehtuuri lyhyesti

Järjestelmä tulee hyödyntämään autentikoinnissa JWT-tokeneita. JWT on avoin standardi tiedon turvalliseen siirtämiseen osapuolten välillä JSON-objektina. Tieto voidaan varmentaa ja siihen

voidaan luottaa, koska se on digitaalisesti allekirjoitettu joko salaisella avaimella (HMAC) tai julkinen/yksityinen avainparilla (RSA tai ECDSA) (jwt-documentaton, 2026). Järjestelmään ei kuitenkaan itse rakenneta omaa JWT-autentikointia vaan käytetään Supabase Authia, joka käyttää sisäisesti JWT:tä, mutta abstrahoi sen devaajalta pois. Itse devaajan en siis joudu hallinnoimaan tokenien luontia, allekirjoitusta tai validointia vaan Supabase hoitaa ne automaattisesti.

HTTPS (Hypertext Transfer Protocol Secure) on HTTP-protokollan salattu versio, joka suojaa asiakkaan ja palvelimen välisen tietoliikenteen. HTTPS käyttää TLS-salausta (Transport Layer Security), joka varmistaa, että siirrettävä data on salattu eikä kolmas osapuoli pysty lukemaan tai muokkaamaan sitä siirron aikana (Cloudfare, 2026). Kirjanpitosovelluksessa HTTPS on kriittinen vaatimus, sillä sovellus käsittelee arkaluonteista taloudellista dataa sekä käyttäjien henkilötietoja. GDPR velvoittaa suojaamaan henkilötiedot asianmukaisin teknisin toimenpitein, ja HTTPS on yksi keskeisimmistä teknisistä suojausmenetelmistä. Käytännössä Supabase ja Vercel tarjoavat automaattisen HTTPS-tuen, joten erillistä SSL-sertifikaatin hallintaa ei tarvita.

5.5 AI-moduulin suunnittelu

Järjestelmän ytimenä toimii AI-moduuli, joka erottaa sen selkeästi muista kirjanpito-ohjelmistoista. AI-moduuli jakautuu kahteen erilliseen komponenttiin: OCR-pipeline sekä LLM-integraatio.

OCR-pipeline on koko sovelluksen ydin. Käyttäjä lataa kuitenkin sovellukseen joko ottamalla kuvan puhelimella tai lataamalla sähköisen PDF-tiedoston. Tiedosto lähetetään Google Vision API:lle, joka tunnistaa kuvasta tekstin OCR-teknologialla ja palauttaa raakatekstin palvelimelle. Koska kuittien rakenne vaihtelee merkittävästi, raakatekstiä ei voida luotettavasti parsia säännöllisillä lausekkeilla, joten teksti välitetään OpenAI GPT-4o-mini -mallille. Malli jäsentää tekstistä strukturoidun JSON-objektin sisältäen myyjän nimen, päivämäärän, kokonaissumman, ALV-erittelyn sekä kategorian. Kategorialuokittelu perustuu mallille syötettyyn listaan 20 ennalta määritellystä kategoriasta, joista malli valitsee sopivimman. Mikäli malli epäonnistuu valitsemaan validin kategorian, järjestelmä palautuu oletusarvoisesti "Muu"-kategoriaan. Malli palauttaa aina

ennalta määritellyn JSON-rakenteen, mikä estää hallusinaatiot ja varmistaa datan yhteensopivuuden tietokannan kanssa.

LLM-integraatiota käytetään myös AI-assistentissa, jonka kautta käyttäjä voi esittää kirjanpitoonsa liittyviä kysymyksiä luonnollisella kielellä. Jokainen pyyntö sisältää käyttäjän kirjanpitoa dataa kontekstina, ja mallille välitetään 20 viimeisimmän viestin keskusteluhistoria, jotta malli pystyy vastaamaan aiempaan kontekstiin pohjautuen. Keskusteluhistoria tallennetaan tietokantaan, joten aiempiin keskusteluihin voi palata myöhemmin. Lisäksi LLM generoi automaattisesti jokaiselle chatille otsikon ensimmäisen viestin perusteella.

5.6 Tietokantasuunnittelu

Tietokannan suunnittelussa käytin dbdiagram.io-ohjelmistoa. Ohjelmisto tarjoaa SQL-tyyppisen syntaksin, jonka avulla taulut ja niiden väliset suhteet voidaan määritellä selkeästi ja tehokkaasti. Kyseinen syntaksi tunnetaan nimellä DBML (Database Markup Language), ja se on suunniteltu erityisesti tietokantojen rakenteen kuvaamiseen yksinkertaisella ja luettavalla tavalla.

Ohjelmiston merkittävä etu on reaaliaikainen visualisointi. Kun tauluja ja suhteita kirjoitetaan DBML-syntaksilla, ohjelmisto päivittää kaavion automaattisesti ja välittömästi. Tämä helpottaa huomattavasti tietokannan rakenteen hahmottamista ja mahdollistaa nopean iteroinnin suunnitteluvaiheessa.

Suunnittelun aikana määriteltiin tietokannan päätaulut sekä niiden väliset relaatiot. Esimerkiksi käyttäjätaulu ja dokumenttitaulu yhdistettiin viiteavaimella, jolloin jokainen dokumentti on sidottu tiettyyn käyttäjään. Tällainen rakenne noudattaa relaatiotietokantojen perusperiaatteita ja varmistaa tiedon eheyden.

Suunniteltu tietokanta koostuu yhdeksästä erilaisesta taulusta. Taulut ovat: users, joka vastaa käyttäjän perusdatan tallentamisesta (etunimi, sukunimi, sposti, salasana, jne.). Documents taulu, joka koostuu ladattujen dokumenttien nimestä, tiedostotyyppistä ja koosta. Documents taulu myös näyttää osoitteen johon tiedosto on tallennettu Supabasessa. Receipts taulu puolestaan viittaa documents tauluun, mutta määrittelee tarkemmin dokumentin sisältöä.

Receipts taulu kertoo dokumentista esimerkiksi myyjän nimen, kokonaissumman, ALV:in, kategorian sekä OCR-tarkkuuden.

Categories taulu, jota käytetään esimerkiksi Receipts taulussa määrittää kuitin kategorian esim. onko se toimistotarvike, matkakulu, jne. Categories tauluun sisällytetään myös tilikoodi sekä onko tietty kuitti ALV-vähennyskelpoinen.

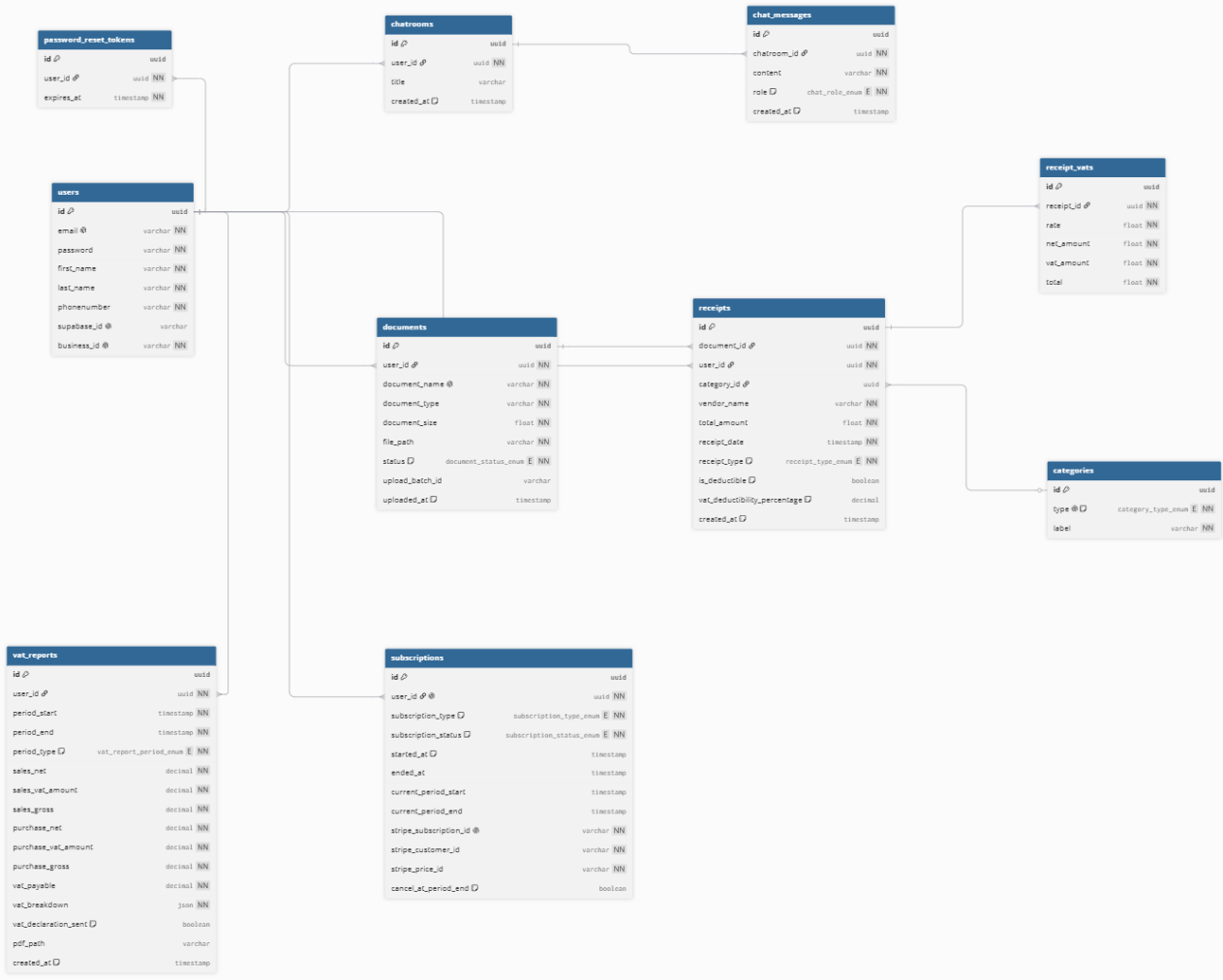
Maksutapahtumiin ja tilauksiin liittyen on suunniteltu myös taulut. Subscriptions taulu viittaa tottakai käyttäjään sekä sisältää tilaus tyypin, joka on määritelty kolmeksi eri vaihtoehdoksi (basic, plus, premium). Taulu sisältää myös päivämäärän tilauksen alkamisesta. Payments taulu, joka viittaa subscriptions tauluun kertoo tarkemmin tilauksesta, esimerkiksi summan, statuksen, maksetut ajankohdat, jne. Payments taulu sisältää tarkempia kohtia itse tilauksesta vähän niin kuin receipts taulu ↔ documents taulu.

Käyttäjän muodostetuilla raporteille on myös suunniteltu taulu. Raporteille on muodostettu omat tyypit, joita ovat tuloslaskelma, alv-yhteenveto sekä kuluraportti. Raportille on myös suunniteltu aikavälikenttä, koska käyttäjä voi valita ohjelmistossa, miltä aikaväliltä haluaa raportin tehdä.

AI-analyysille ja AI chatti viesteille on myös suunniteltu vastaavat taulut. Chatti viestien taulu näyttää esimerkiksi session_id:n tiettyyn chatiin sekä sen sisällön. AI-analyysi puolestaan viittaa useriin sekä documenttiin. AI-analyysille on luotu kolme mahdollista tyyppiä, joita ovat: kuitin tietojen poiminta, automaattinen tiliöinti sekä poikkeamien tunnistus.

Tietokanta on karkeasti suunniteltu ja tulee täysin varmasti muokkautumaan ja kehittymään itse kehityksen sprinttien edetessä. Suunnittelun ideana on kuitenkin luoda peruspohja ja idea siitä miltä valmiin ohjelmiston tietokanta tulee näyttämään. Luulen, että suunniteltu tietokantadokumentaatio auttaa hahmottamaan järjestelmän kokonaisarkkitehtuuria ja toimii hyvänä lähtökohtana kehitystyölle.

Alla on päivitetty kuva valmiista tietokannasta.



6 Testaussuunnitelma (DSR vaihe 3)

6.1 Testausstrategia

Projektin aikana testaus on prosessi, jota tullaan tekemään jokaisen sprintin aikana. Erilaisia testausstrategioita ja menetelmiä on olemassa useita ja näitä tullaan hyödyntämään monipuolisesti projektin edetessä. Karkeasti sanottuna projektin testaus tulee jakautumaan viiteen eri osa-alueeseen, joita ovat yksikkötestaus, integraatiotestaus, end-to-end testaus, AI-mallin arviointi sekä suorituskyykytestausta.

Yksikkötestauksella tullaan testaamaan ohjelmiston yleisimpiä komponentteja sekä yksittäisiä funktioita. Tavoitteena on varmistaa, että jokainen yksittäinen osa toimii oikein itsenäisesti ennen kuin se integroidaan osaksi laajempaa kokonaisuutta. Next.js-frontendissa yksikkötestaus kohdistuu yksittäisiin React-komponentteihin ja Node.js-backendissa yksittäisiin API-funktioihin ja apufunktioihin.

Integraatiotestaus puolestaan tarkoittaa, että testataan kuinka eri kerrokset toimivat keskenään. Esimerkkinä vaikka kuinka backend-puolella kahden eri routin kommunikointi toimii keskenään, tai kuinka backend kommunikoi tietokannan kanssa Prisma ORM:n kautta.

Integraatiotestauksessa varmistetaan myös ulkoisten palveluiden, kuten Stripe-maksunvälittäjän ja Google Vision API:n integraatioiden toimivuus.

End-to-end testausta puolestaan hyödynnetään, kun halutaan simuloida kokonainen käyttäjäpolku alusta loppuun. Esimerkiksi käyttäjä rekisteröityy järjestelmään, kirjautuu sisään, lataa kuitin, hyväksyy OCR:n tunnistamat tiedot ja generoi lopuksi ALV-yhteenvedon. Tämä testausmuoto on erityisen tärkeä kirjanpitosovelluksessa, koska virhe missä tahansa vaiheessa käyttäjäpolkua voi johtaa virheellisiin kirjanpitomerkintöihin.

AI-mallin arviointi on tämän tutkimuksen kannalta kriittisin testauksen osa-alue, sillä se vastaa suoraan asetettuihin tutkimuskysymyksiin. OCR-tunnistuksen tarkkuutta arvioidaan testaamalla järjestelmää useilla kymmenillä erityyppisillä kuiteilla ja laskemalla kuinka suuri prosentti

summista, päivämääristä ja myyjien nimistä tunnistetaan oikein. Tavoitteena on saavuttaa vähintään 95 % tarkkuus selkeistä kuiteista. Automaattisen tiliöinnin oikeellisuutta arvioidaan puolestaan vertaamalla AI:n ehdottamaa kategoriaa oikeaan tiliöintiin testiaineistolla.

Suorituskykytestauksessa varmistetaan, että sovellus täyttää ei-toiminnalliset vaatimukset suorituskyvyn osalta. Testattavia kohteita ovat muun muassa sivujen latausajat, ALV-yhteenvedon ja tuloslaskelman generointinopeus sekä järjestelmän toiminta samanaikaisten käyttäjien kuormituksessa.

7 Toteutus (DSR vaihe 3)

Luvussa käydään läpi ohjelmiston toteutusta, teknologiavalintoja ja perusteluja niille sekä ongelmia mitä ohjelmistoa tehdessä ilmeni. Luvun tarkoituksen on myös kuvata, millaista ohjelmistokehitys oli kehittäjälle kehitysvaiheessa sekä millainen ajatusprosessi oli.

7.1 Teknologiavalinnat ja perustelu

Teknologiavalinnat olivat alusta alkaen suhteellisen selkeät. Koko ohjelmisto kehitettiin TypeScriptillä, joka mahdollistaa sulavaa web kehitystä sekä ennen kaikkea tyyppitystä, joka mahdollistaa muun muassa vihreiden havaitsemisen jo kehitysvaiheessa ilman, että nämä virheet päätyvät tuotantoon. Sekä omasta mielestäni TypeScript tarjoaa esimerkiksi JavaScriptiin verraten paremman koodin ylläpidettävyyden sekä luettavuuden, mikä on tärkeää laajoissa ohjelmistoissa.

Ohjelmisto toteutettiin Next.js, Express, Prisma ORM pinolla. Client eli frontend puolen viitekehyksenä toimi Next.js, server puolen eli backend puolen viitekehyksenä oli puolestaan Express, jota ajettiin Node.js ympäristössä ja tietokantana toimi Postgre SQL, jota käytettiin Prisma ORM työkalun avulla sen sijaan, että kirjoitettaisiin SQL-syntaksia suoraan tiedostoihin.

Next.js ja Express olivat loogisia valintoja, koska kyseessä oli kuitenkin web kehityksen projekti sekä molemmat viitekehykset ja niiden ympäristöt käyttivät samaa ohjelmointikieltä. Tietokanta valinta oli myös helppo valinta. Postgre SQL toimii erinomaisesti relaatiotietokantana sekä siihen valittu Prisma ORM oli erinomainen ratkaisu, sillä se mahdollisti tietokantakyselyjen kirjoittamisen suoraan TypeScriptä-syntaxilla ilman SQL:n kirjoittamista tiedostoihin. Prisma ORM ohelle valitsin myös Supabasen. Supabasen avulla pystyttiin helposti esimerkiksi muodostamaan JWT tokeneita käyttäjien autentikointiin sekä tiedostojen tallentaminen tänne oli erityisten kätevää.

Testaus kirjastot mitä projektissa käytettiin, olivat Jest server puolen unit- sekä integraatiotestauksiin. Cypress testaus kirjasto puolestaan E2E-testeihin, missä simuloitiin

kokonaisia käyttäjä polkuja esimerkiksi: käyttäjän luonti -> kirjautuminen -> tiedostonlataus. Tekoäly ohjelmointirajapinnoiksi valittiin Google vision API, joka mahdollisti OCR-teknologian. OpenAI API, joka puolestaan vastasi OCR:llä eritetyn datan järjestelemisestä ja analysoinnista.

Muita kirjastoja ja työkaluja, mitä projektissa käytin olivat Redis + BullMQ, Stripe maksujärjestelmä, Resend sähköpostiviestien lähetykseen, Winston yleiseen loggaukseen sekä Zod tyyppitykseen ja validointiin.

7.2 AI-mallin valinta

Alusta asti oli selkeää, että projektiin tarvittiin kaksi eri tekoälypohjaista ohjelmointirajapintaa. Sovelluksen logiikan idea oli, että käyttäjä lataa tiedoston järjestelmään (kuva tai PDF), OCR-teknologia eli Optical Character Recognition tunnistaa kuvastaa merkit, kirjaimet ja numerot ja antaa vastauksena nämä palvelimelle. Mutta, koska kuitit voivat olla muodoltaan hyvin erilaisia toisistaan, ei ole varmuutta, että palautettu vastaus olisi aina muodossa: Myyjän nimi, summa, ALV, jne. Tätä ei myöskään voi Regexillä oikeastaan mitenkään voi jäsenellä, koska kuitit voivat olla hyvin erilaisia toisistaan. Ainoa ratkaisu oli syöttää OCR antama data jollekin LLM:lle esim. OpenAI API:lle. LLM pystyy hyvällä promptilla antamaan datan halutussa muodossa esim. myyjän nimi, summa ja ALV.

Google Vision API toimi hyvin OCR ratkaisuna, kun taas OpenAI API puolestaan toimi hyvin LLM:nä. Google Vision API tarjosi kolmen kuukauden ilmaisen kokeilun ja sen antamat "credentialit" oli helppo käyttää. OpenAI API puolestaan vaati maksun, mutta esim. 5 dollarilla saa pyyntöjä enemmän kuin tarpeeksi. OpenAI consolen sivuilta myös sai varmistettua, että syötettyä dataa ei käytetä LLM:n jatkokoulutukseen, mikä oli GDPR:n kannalta oleellista.

OpenAI API:lle on muodostettu ohjelmistossa kolme eri funktiota, joissa kussakin on yksi prompti. Nämä funktiot vastaavat muun muassa kuitti datan parsimisesta, käyttäjän kysymyksiin vastaamisesta sekä chatin "titlen" muodostamisesta. Mallina käytetään gpt-4o-mini, koska se on erittäin edullinen kustannuksien kannalta sekä olen todennut, että sen antamat vastauksen ovat

erittäin tarkkoja. Kuitti datan parsiminen -funktio määrittelee tarkan JSON rakenteen, mitä AI:n tulee palauttaa, mikä estää tekoälyn hallusinoitua ja oman datan keksimistä. Kategorialista myös injektoidaan funktioon dynaamisesti, jolloin AI:n annetaan valita 20 kategorian väliltä sopiva kategoria. Mikäli kuitenkin AI epäonnistuisi valitsemaan validi kategoria niin palataan defaulttiin MUU kategoria vaihtoehtoon. Chatti viestien muodostamisen kutsu puolestaan ottaa 20 viestin chat historian osaksi promptia, jotta AI osaa nähdä kontekstin, mistä käyttäjän kanssa on puhuttu.

7.3 Backend-toteutus

Server puolen eli backend puolen sovelluskehikseksi valittiin Express, jota käytettiin Node.js ympäristössä. Expressistä minulla oli aiemmin paljon kokemusta, jota valinta oli erinomainen. Osasin muodostaa organisoidun kokonaisuuden heti alusta alkaen jaotellen backendin erilaisiin kokonaisuuksiin kuten routeihin, controllereihin, serviceihin, testeihin, middlewareihin, jne.

Controllerit sisältävät funktiot, jotka käsittelevät kunkin reitin (route) pyynnöt ja vastaukset. Nämä funktiot yhdistetään Routeriin routes-tiedostoissa, ja Routerit otetaan lopulta käyttöön app.ts-tiedostossa. Moneen routtiin on yhdistetty erilaisia middlewareja, joista yleisimmät ovat autentikaatio sekä "rate limiter".

Autentikaatio perustuu Supabasessa muodostettuun JWT-autentikointitokeniin. Ohjelmistoon kirjautumisen yhteydessä käyttäjän tietoja vasten luodaan JWT-token, joka lähetetään evästeeseen (cookie). Tämä eväste lähetetään automaattisesti jokaisen pyynnön mukana palvelimelle, jossa token validoidaan ja käyttäjän identiteetti varmistetaan ennen kuin pyyntö sallitaan käsiteltäväksi. Autentikaatio middlewarea käytetään lähes kaikissa routeissa lukuun ottamatta tietenkin esim. signup, login ja password-reset.

Rate limiter middlewaren idea on puolestaan rajoittaa yksittäisen käyttäjän pyyntöjen määrä, joka estää jossain määrin palvelimen ylikuormitusta sekä Ddos hyökkäyksiä. Rate limiter perustuu joko käyttäjän ID:hen tai IP-osoitteeseen. Kaikki routit, joissa käyttäjä on jo kirjautunut, voidaan hyvin käyttää user_id:tä, mutta puolestaan routit kuten login ja signup suojataan IP-

osoitteeseen kohdistuvan rajoittimen avulla. Rate limiterin ideana on, että jokaiselle routelle tulee oma rate limiter "bucket" jossa on tietty määrä pyyntöjä, mitä minuutissa saa tehdä. Tämän rajan ylittyessä tulee lukitus, jossa käyttäjälle ilmoitetaan, eikä hän voi lukituksen aikana tehdä pyyntöjä **tähän** osoitteeseen. Rate limitereitä on ohjelmistossa käytännössä useampi kuin yksi. Esimerkiksi raskaammat operaatiot kuten tiedoston upload käyttävät eri rate limiteriä kuin useammat GET routit. Raskaammissa operaatioissa on vähemmän sallittuja pyyntöjä per minuutti ja rate limiterit on jaoteltu esim. `rate_limiter.read()`, `rate_limiter.write()`, `rate_limiter.heavy_read()`, `rate_limiter.heavy_write()` tyyliin, jossa sulkeisiin syötetään vielä bucketin nimi.

Tiedostojen käsittely on toteutettu asynkronisesti BullMQ-jonon avulla, joka käyttää Redistä taustajärjestelmänä. Kun käyttäjä lataa kuitin, tiedosto tallennetaan välittömästi Supabase-tallennustilaan ja tietokantaan luodaan dokumenttitietue tilalla PENDING. Tämän jälkeen käsittelyjob lisätään jonoon, ja API vastaa käyttäjälle välittömästi ilman, että sen tarvitsee odottaa kuitin analysointia.

Erillinen worker-prosessi lukee jonosta jobeja ja käsittelee ne yksi kerrallaan. Worker lataa tiedoston Supabasesta, analysoi sen Google Vision OCR:llä tekstiksi, jonka jälkeen OpenAI jäsentää tekstistä strukturoidun datan, myyjän nimen, päivämäärän, summan, ALV-erittelyn sekä kategorian. Onnistuneen käsittelyn jälkeen dokumentin tila päivitetään COMPLETED-tilaan ja kuittitiedot tallennetaan tietokantaan.

Jonon käyttämisen etuna on se, että raskas käsittely ei blokkaa HTTP-pyyntöä. Eli AI analysointi toimii käytännössä ns. taustaprosessina, jossa käyttäjä voi tehdä muutakin ohjelmistossa, sillä välin, kun analysointi on meneillään. Jos käsittely epäonnistuu esimerkiksi API-virheen takia, BullMQ yrittää jobin uudelleen automaattisesti eksponentiaalisella viiveellä. Käyttäjä voi seurata käsittelyn tilaa erillisen status-endpointin kautta, johon frontend tekee pollauksia. Pollauksia tehdään myös kuittien haku sivuilla, mikäli joku kuitti sisältää PENDING tilan ja kuitit näytetään käyttäjälle reaaliaikaan niiden analysoitua. Kun yksikään dokumentti ei sisällä PENDING tilaa pollaus lopetetaan.

Ohjelmistossa toimii Stripe maksujärjestelmä. Stripe on konfiguroitu ottamaan vastaan vuosittaisia ja kuukausittaisia basic ja premium tilauksia. Ohjelmistossa luotu checkout flow toimii periaatteessa näin: Kun käyttäjä valitsee tilauksen, luodaan Stripe Checkout -sessio jossa määritellään hinta, onnistumis- ja peruutus-URL sekä metadata (user_id, subscriptionType). Käyttäjä ohjataan Stripen omalle maksusivulle, jolloin sovellus ei itse käsittele maksukorttitietoja. Ohjelmistossa on myös webhook pohjainen tilauksen hallinta, joka kuuntelee stripe pyyntöjä ja tekee sitä mukaa tilauksia, peruutuuksia ja peruutuksen perumisia.

Tilauksen tila ei päivity suoraan checkoutin jälkeen vaan Stripe lähettää webhook-tapahtumia palvelimelle eri tilanteissa, esimerkiksi checkout.session.completed, invoice.payment_succeeded, customer.subscription.deleted ja customer.subscription.updated. Palvelin käsittelee nämä tapahtumat ja päivittää tietokantaan tilauksen tilan vastaavasti. Tämä on luotettavampi tapa kuin luottaa pelkkään redirect-URLiin, koska webhook tulee aina Stripeltä riippumatta siitä sulkeeko käyttäjä selaimen kesken maksun.

Testauksessa käytetään backend puolella Jestia unit- ja integraatiotestaukseen. Jestin avulla on muodostettu 180 testiä yhteensä, joka kattaa kaikki ohjelmiston kontrollerit ja serviceiden (Open AI, OCR, jne.) ja middlewarejen unit testit. Testeissä saavutettiin testikattavuudeksi 89.54 %, lausekattavuudeksi 75.4 % ja haarakattavuudeksi 90.35 %. Ulkoiset riippuvuudet, kuten tietokantakutsut, Stripe ja Supabase, on korvattu testeissä mock-objekteilla, jolloin testit ovat nopeita ja eivät vaadi verkkoyhteyttä. Testit ajetaan myös erillisessä Docker-testiympäristössä, jossa on oma PostgreSQL-tietokanta, jolloin testit eivät vaikuta tuotanto- tai kehitysympäristön dataan.

7.4 Frontend-toteutus

Sovelluskehikseksi eli "frameworkiksi" valittiin frontend puolella Next.js. Next.js sovelluskehiksenä oli hieman tuntematon itselleni projektin alkuvaiheessa. Reactia olen käyttänyt useisiin projekteihin kuitenkin, niin oppimiskaari oli suhteellisen nopea. Mikä teki Next.js:stä erityistä oli sen ominaisuus käyttää server-komponentteja, server action tiedostoja sekä client komponentteja. Se miten itse ymmärsin esimerkiksi server-komponentin ja client

komponentin eron Next.js:ssä oli, että server-komponentti renderöidään valmiiksi jo palvelimella eikä se lähetä komponentin omaa JS-koodia selaimelle, mikä tarkoittaa pienempää "bundlekokoa" eli toisin sanoen kevyempää komponenttia. Server komponentissa puolestaan ei ole mahdollista käyttää Reactille tuttuja `useState`, `useEffect`, `useRef`, yms. vastaavia toimintoja. Datan haku esimerkiksi palvelimelta on puolestaan kätevää. Voidaan suoraan asettaa komponentti asynkroniseksi ja tehdä ja hakea käytännössä yhdellä rivillä data suoraan muuttujaan `await` avainsanalla sen sijaan että pitäisi käyttää `useEffect`-hookkia.

Client komponentin saa puolestaan muodostettua käyttämällä 'use-client' merkkijonoa tiedostossa. Client komponentti mahdollistaa puolestaan sen, että komponentti lähettää JS-koodin selaimelle. Client komponentissa ensin renderöidään HTML ja sitten "herätetään HTML eloon" selaimessa lisäämällä interaktiivisuutta sivuun esimerkiksi edellä mainituilla `useState`, `useEffect` tai `useRef` toiminnoilla. Jokaista komponenttia ei kuitenkaan automaattisesti voi leimata "use-client" merkkijonolla sillä, client komponentit ovat raskaampia, koska ne kasvattavat lähetettävän JavaScripti-bundlen kokoa ja vaatii "hydratoinnin", mikä lisää selaimen suoritustaakkaa verrattuna server komponentteihin. Tässä tuleekin tilanne, jossa joutui tarkkaan miettimään sivun ja ohjelmiston arkkitehtuuria. Mitkä komponentit oltava client-komponentteja ja mitkä taas server-komponentteja. Myös täytyi muistaa, että jos "parent komponentti" on client-komponentti, niin sen child komponentti on myös aina client-komponentti. Eli miettien tätä hierarkiaa, server-komponenttien on oltava ylempänä hierarkiassa puussa.

Next.js oli myös rakenteeltaan erittäin looginen ja kaikki tiedostot ja komponentit pysyivät järkevästi organisoituina. (main) kansion sisään tulivat suurin osa sovelluksen reiteistä (/upload, /dashboard, /receipts, tms.). Yksi pääsivu koostuu aina yhdestä kansiosta kuten /receipts. Kansion sai aina organisoidusti jaoteltua `Page.tsx`, komponentteihin sekä `action.ts` tiedostoihin. `Page.tsx` on käytännössä se reitin ydinosa sekä tässä "hierarkia puussa" se ylin komponentti. Yleensä muodostetaan server komponenttina ja importataan mahdolliset kentät palvelimelta ja tietokannasta antaen nämä "propseina" child komponenteille. `Action.ts` tiedostot sisältävät puolestaan kaikki backend "fetch pyynnöt", joissa tehdään http-metodeilla eli GET, POST, PUT ja DELETE pyyntöjä backendin routteihin. `Action.ts` tiedostoissa funktiot säilyvät organisoituina sekä

mahdollistaa myös niiden uudelleenkäytön ohjelmistossa sen sijaan, että ne olisi suoraan toteutettu komponentteihin.

7.5 Kehityshaasteet ja ratkaisut

Kehityshaasteita devatessa tuli paljon, koska se on osa kehitysprosessia. Yksi merkittävimmistä teknisistä haasteista oli kuittianalysoinnin asynkroninen toteutus. Alun perin ajattelin toteuttaa analysoinnin suoraan HTTP-pyynnön sisällä, mutta tämä olisi aiheuttanut ongelmia pitkien vasteaikojen ja aikakatkaisujen muodossa, sillä Google Vision OCR ja etenkin OpenAI-kutsut voivat kestää useita sekunteja. Ratkaisuna otettiin käyttöön BullMQ-jono Redistä taustajärjestelmänä, jolloin analysointi siirrettiin erilliselle worker-prosessille. Tämä mahdollisti välittömän vastauksen käyttäjälle ja teki järjestelmästä huomattavasti luotettavamman, sillä epäonnistuneet jobat yritetään automaattisesti uudelleen.

Tekoälyassistentin vastausten suoratoisto (streaming) osoittautui haastavaksi toteuttaa. Tavallinen request-response -malli ei sovellu tilanteeseen, jossa palvelimen pitää lähettää dataa jatkuvana virtana asiakkaalle. Ratkaisuna käytettiin Server-Sent Events -tekniikkaa, jossa palvelin pitää HTTP-yhteyden auki ja lähettää jokaisen OpenAI:lta saapuvan tekstikappaleen välittömästi selaimelle. Tämä loi käyttäjälle luontevan kokemuksen, jossa vastaus ilmestyy ruudulle sana kerrallaan.

Kirjanpitosovellus käsittelee arkaluonteista taloudellista dataa, joten tietosuojavaatimusten selvittäminen oli aikaa vievää. Erityisen haastavaa oli selvittää OpenAI:n datan käsittelykäytännöt ja se, täyttääkö palvelu GDPR:n vaatimukset eurooppalaisten käyttäjien osalta. Tietosuojaselosteen ja käyttöehtojen kirjoittaminen ilman juridista asiantuntemusta oli vaativaa, ja nämä dokumentit vaatisivat jatkossa lakimiehen tarkistuksen.

Projekti osoittautui suhteellisen laajaksi yhdelle kehittäjälle. Full-stack sovelluksen rakentaminen sisältäen frontendin, backendin, tietokannan, ulkoiset integraatiot, testauksen, tietoturvan ja dokumentaation vaati jatkuvaa priorisointia. Tähän ei selkeää ratkaisua ollut vaan se vaati pitkäjänteisyyttä ja ennen kaikkea ajan priorisointia, jotta projektin sai tehtyä deadline

mukaisesti. Käytin myös iteratiivista kehitysprosessia GitHub Issueiden ja sprinttien avulla, jolloin kehitys eteni hallitusti ominaisuus kerrallaan.

7.6 Testaus

Ohjelmiston testaus toteutettiin backend-puolella Jest-testauskirjastolla. Testit kattavat yhteensä 19 testitiedostoa ja 180 yksittäistä testitapausta, jotka kaikki läpäistiin onnistuneesti. Testit ajetaan erillisessä Docker-pohjaisessa PostgreSQL-testitietokannassa, jotta tuotantodataan ei kohdistu muutoksia testiajon aikana. Tietokanta alustetaan ennen jokaista testiajoa ja tyhjennetään ajon jälkeen, mikä takaa testien toistettavuuden ja riippumattomuuden toisistaan.

Test Suites: 19 passed, 19 total
Tests: 180 passed, 180 total

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	89.54	75.4	90.35	89.2	
src	100	50	100	100	
app.ts	100	50	100	100	34-62
src/controllers	87.06	79.92	93.24	86.38	
assistant.controller.ts	89.33	71.42	100	89.55	34, 98, 112, 120, 136, 150, 159
cypress.controller.ts	19.14	0	0	15.9	17-74, 84-116
dashboard.controller.ts	92.59	100	100	92	63, 123
login.controller.ts	86.36	83.33	100	85.71	24, 50-51
logout.controller.ts	100	100	100	100	
password-reset.controller.ts	90.74	83.33	100	90.38	56, 100, 104, 115, 125
receipt.controller.ts	92.3	80.88	90.9	91.8	75, 105, 163, 199-200, 210, 249, 285, 321, 418
report.controller.ts	96.82	100	100	96.55	150, 167, 197, 222, 249, 378
signup.controller.ts	67.56	70	50	65.71	50, 75-77, 90-100
storage.controller.ts	89.02	78.12	100	88.46	85, 91-92, 100, 131, 139, 160, 191, 205
subscription.controller.ts	89.55	87.5	100	88.7	33, 48, 72, 111, 130, 162, 181
user.controller.ts	85.88	78.26	100	84.81	31-32, 62, 67, 99, 123, 133, 145, 155, 162, 170, 203
webhook.controller.ts	100	70	100	100	61-62
src/lib	96.29	45.83	100	96.29	
logger.ts	100	75	100	100	5
prisma.ts	90.9	35.71	100	90.9	12
redis.ts	100	50	100	100	3-7
supabase.ts	100	50	100	100	6
src/middleware	96.77	92	100	98.21	
authentication.ts	100	100	100	100	
errorHandler.ts	87.5	50	100	85.71	24
rateLimiterMiddleware.ts	91.66	75	100	100	7
requestLogger.ts	100	100	100	100	
subscription.ts	100	100	100	100	
src/queues	100	100	100	100	
queue.ts	100	100	100	100	
src/routes	100	100	100	100	
assistant.routes.ts	100	100	100	100	
cypress.routes.ts	100	100	100	100	
dashboard.routes.ts	100	100	100	100	
login.routes.ts	100	100	100	100	
logout.routes.ts	100	100	100	100	
password-reset.routes.ts	100	100	100	100	
receipt.routes.ts	100	100	100	100	
report.routes.ts	100	100	100	100	
signup.routes.ts	100	100	100	100	
storage.routes.ts	100	100	100	100	
subscription.routes.ts	100	100	100	100	
user.routes.ts	100	100	100	100	
webhook.routes.ts	100	100	100	100	
src/schemas	100	100	100	100	
auth.schema.ts	100	100	100	100	
chat.schema.ts	100	100	100	100	
id.schema.ts	100	100	100	100	
receipt.schema.ts	100	100	100	100	
report.schema.ts	100	100	100	100	
subscription.schema.ts	100	100	100	100	
src/services	78.37	62.16	58.33	77.14	
mailer.service.ts	91.66	80	100	91.66	23
ocr.service.ts	77.77	77.77	50	76.47	8-10, 23
openai.service.ts	72.72	37.5	33.33	68.42	13-33, 43-58, 68
stripe.service.ts	100	100	100	100	
supabase.service.ts	75	50	100	75	14, 18, 34, 46, 50
src/tests/helpers	93.75	75	100	93.75	
createReceipt.ts	100	100	100	100	
createUser.ts	91.66	75	100	91.66	18
src/utls	94.73	36.36	93.33	94.73	
error.ts	92.3	12.5	87.5	92.3	51-52
rateLimiter.ts	100	100	100	100	

Kokonaisrivikattavuus on 89,2 % ja lausekattavuus 89,54 %. Reittitiedostot (routes), validointiskemat (schemas) sekä middleware-kerros saavuttivat 100 % kattavuuden kaikissa

mittareissa. Kontrolleritiedostojen kattavuus vaihtelee välillä 86–100 %, poikkeuksena Cypress-testejä varten tehty testiapuohjain (`cypress.controller.ts`), jota ei ole tarkoitettu tuotantokäyttöön eikä sen testaaminen ole oleellista. Palvelukerroksen (`services`) kattavuus on matalampi johtuen siitä, että kolmansien osapuolten rajapintoja (Google Vision API, OpenAI) kutsuvaa koodia ei yksikkötesteissä ajeta oikeasti, vaan ne ovat mockattu, joten osa koodihaaran logiikasta jää siten kattavuusmittauksen ulkopuolelle.

Haarautumakattavuus (`branch coverage`) kertoo, kuinka suuri osa koodin haarautumisista on testattu. Haarautuminen tarkoittaa kaikkia kohtia, joissa koodi voi kulkea eri suuntiin. Tyypillisesti `if/else`-rakenteet, `switch`-lauseet sekä ternary-operaattorit. Käytännön esimerkkinä yksittäinen reitti voi palauttaa useita eri vastauksia tilanteen mukaan: 401 autentikointivirhe jos käyttäjä ei ole kirjautunut, 400 validointivirhe jos pyyntödata on virheellistä, 404 jos haettua resurssia ei löydy, tai 200 onnistumiskoodi sekä varsinainen data onnistuneen kutsun tapauksessa. Jokainen näistä vastausvaihtoehdoista muodostaa oman haaransa, ja korkea haarautumakattavuus edellyttää, että testit kattavat kaikki nämä skenaariot. Suurin osa controllereiden funktioista saavuttaa yli 80 % `branch coverage`n, mikä on suhteellisen hyvä.

Frontend-puolella testaus toteutettiin Cypressillä end-to-end-testeillä, jotka kattavat sovelluksen keskeisimmät käyttäjäpolut: rekisteröitymisen, kirjautumisen, kuittien latauksen ja tarkastelun sekä AI assistentin kokeilun. End-to-end-testit ajetaan täyttä sovelluspinoa vasten, jolloin frontend, backend ja tietokanta ovat kaikki käynnissä testiajon aikana.

7.7 GDPR

GDPR eli General Data Protection Regulation on Euroopan Unionin yleinen tietosuojalake. Se yhtenäistää tietosuojalakeja ja velvoittaa kaikkia EU-kansalaisten henkilötietoja käsitteleviä organisaatioita. Asetus antaa yksilöille vahvan kontrollin omiin tietoihinsa ja vaatii yrityksiltä läpinäkyvyyttä, turvallisuutta ja vastuullisuutta (Euroopan Unioni, 2025).

Koska minun projektini käsitteli jossain määrin henkilötietoja (kuitteja), olivat GDPR käytänteet koko ajan läsnä työskennellessä. GDPR:n mukaan yrityksen ja organisaatioiden on noudatettava

GDPR:n periaatteita, joita ovat muun muassa lainmukaisuus, osoitusvelvollisuus sekä riskienhallinta. Lainmukaisuus tarkoittaa, että tietojen käsittelylle on aina oltava peruste kuten sopimus, suostumus tai lakisääteinen velvoite. Osoitusvelvollisuus puolestaan tarkoittaa, että yritysten on pystyttävä osoittamaan noudattavansa tietosuojaperiaatteita, mikä vaatii usein dokumentointia. Riskienhallinta taas tarkoittaa, että tietoturvan on oltava kunnossa, ja henkilötietojen tietoturvaloukkauksista on ilmoitettava valvontaviranomaiselle ja asianosaisille nopeasti (Euroopan Unioni, 2025).

Sovelluksen kannalta merkittävimmät GDPR-velvoitteet liittyvät tietosuojaselosteen ja käyttöehtojen laatimiseen, käyttäjän oikeuteen poistaa omat tietonsa sekä siihen, että OpenAI-rajapintaan lähetetty data käsitellään Yhdysvalloissa. OpenAI ei käytä rajapintadataa mallien kouluttamiseen oletusarvoisesti, mutta datan siirto EU:n ulkopuolelle on GDPR:n näkökulmasta huomionarvoinen seikka, joka tulisi käytännön sovelluksessa ratkaista asianmukaisella tiedonsiirtomekanismilla.

7.8 OWASP Top 10, Audit tulokset sekä haavoittuvuudet

OWASP (Open Web Application Security Project) Top 10 on laajalti tunnustettu lista kriittisimmistä web-sovellusten tietoturvariskeistä. Se toimii de facto -standardina web-sovellusten tietoturvan arvioinnissa ja kehittämisessä. Alla on käyty läpi jokainen kategoria suhteessa ohjelmiston toteutukseen.

1. Broken Access Control (Rikkinäinen pääsynhallinta)

Tämä on OWASP Top 10 -listan yksi vakavimmista uhkista. Auditoinnin yhteydessä havaittiin useita IDOR-haavoittuvuuksia (Insecure Direct Object Reference), joissa tietokantakyselyt hyväksyivät käyttäjän antaman ID:n ilman omistajuuden tarkistusta. Haavoittuvuudet korjattiin lisäämällä `user_id: req.user.id` -ehto kaikkiin kyselyihin, jotka käsittelevät käyttäjäkohtaista dataa.

2. Cryptographic Failures (Salausvirheet)

Salasanat hashataan bcrypt-kirjastolla, JWT-token tallennetaan httpOnly-evästeeseen eikä selaimen localStorage-tallennustilaan, ja HTTPS on pakollinen tuotantoympäristössä.

3. Injection (Injektiohyökkäykset)

Prisma ORM estää SQL-injektiohyökkäykset parametrisoiduilla kyselyillä. Kaikki käyttäjän syötteet validoidaan myös Zod-skeemoilla ennen käsittelyä.

4. Insecure Design (Epäturvallinen suunnittelu)

Tilaussuojaus on toteutettu requireSubscription-middlewarella, joka suojaa kaikki maksulliset ominaisuudet. Rate limiting on myös käytössä kaikilla reiteillä.

5. Security Misconfiguration (Tietoturvakonfiguraatiovirheet)

Evästeet on konfiguroitu httpOnly, secure ja sameSite: strict -lipuilla. Ympäristömuuttujat on erotettu koodista .env-tiedostoon eikä niitä ole tallennettu versionhallintaan.

6. Vulnerable and Outdated Components (Haavoittuvat ja vanhentuneet komponentit)

npm audit suoritettiin sekä frontend- että backend-riippuvuuksille. Tuotantoriippuvuuksissa ei havaittu haavoittuvuuksia. Kehitystyökaluissa havaittiin 10 kohtalaisen vakavuuden haavoittuvuutta, jotka sijaitsevat better-docs-, @prisma/dev- ja exceljs-paketeissa. Nämä eivät ole läsnä tuotantoympäristössä.

7. Identification and Authentication Failures (Tunnistautumisvirheet)

Autentikaatio on toteutettu Supabasen JWT-tokenin avulla. Salasanat hashataan bcryptillä, sessio mitätöidään uloskirjautumisen yhteydessä ja salasanan palautus käyttää määräaikaista kertakäyttöisiä tokeneita.

8. Software and Data Integrity Failures (Ohjelmiston ja datan eheysongelmat)

Stripe-webhook-tapahtumat verifioidaan allekirjoituksella `stripe.webhooks.constructEvent()` -funktiolla ennen käsittelyä, jolloin väärennetyt webhook-kutsut estetään.

9. Security Logging and Monitoring Failures (Lokituksen ja monitoroinnin puutteet)

Winston-kirjastolla toteutettu lokitus kattaa kaikki routteihin tulleet pyynnöt sekä onnistumiset (2xx) ja virheet (4xx). Myös kaikki muu logataa Winston-kirjaston avulla kuten workereiden toiminta.

10. Mishandling of Exceptional Conditions (Poikkeustilanteiden virheellinen käsittely)

Sovelluksessa on käytössä keskitetty virheenkäsittelijä Express-middlewarena, joka kaappaa kaikki käsittelemättömät virheet ja palauttaa standardoidun virhevastauksen. Kontrollerit käyttävät `next(error)` -kutsua virheiden välittämiseen, eikä sisäisiä virheviestejä tai stack traceja palauteta käyttäjälle tuotantoympäristössä. Ulkoisten palveluiden, kuten Stripen laskutushistorian, virheet käsitellään erillisesti niin, että yksittäinen epäonnistunut kutsu ei kaada koko pyyntöä.

8 Demonstrointi ja arviointi (DSR vaiheet 4 ja 5)

Demonstrointivaihe toteutettiin lataamalla useita erillisiä kuittidatasettejä sovellukseen. Kuitit olivat peräisin ravintola-, kauppa- ja palvelukuiteista. Jokainen kuitti ladattiin sovellukseen, jonka jälkeen järjestelmä suoritti automaattisesti OCR-tunnistuksen Google Vision API:n avulla ja jäsensi tiedot OpenAI GPT-4o-mini -mallilla.

Arviointivaiheessa mitattiin sekä suorituskkyky että tarkkuus. Tarkkuus laskettiin yhtenäisellä kaavalla: jokaisessa kuitissa on kuusi tärkeää kenttää (myyjän nimi, päivämäärä, kokonaissumma, verokanta, veroton summa ja ALV-summa). Jokainen kenttä pisteytetään oikein tai väärin (1 tai 0) ja tulos jaetaan kenttien kokonaismäärällä.

Ensimmäisessä testissä ladattiin datasetti, joka sisälsi 100 normaalia kuittia. Analysointi kesti 7 minuuttia ja 3 sekuntia, eli keskimäärin noin 4 sekuntia per kuitti. Tarkkuudeksi mitattiin 99,9 %, ja ainoat kaksi virhettä koostui ALV-summan pyöristyseroista yksittäisissä kuiteissa.

Kirjanpitäjälle saman datamäärän manuaalinen kirjaaminen veisi arviolta 33–67 minuuttia (20–40 sekuntia per kuitti), minkä lisäksi manuaaliseen työhön liittyy inhimillisten virheiden riski.

Tarkkuuden mittaamiseen kehitettiin erillinen testausmenetelmä. Jokaisesta testidatasetistä muodostettiin JSON-tiedosto, joka sisälsi kuittien oikeat tiedot manuaalisesti tarkistettuna. Tietokantaan analysoitujen kuittien tulokset haettiin automaattisesti TypeScript-skriptillä, joka vertasi jokaisen kuitin kenttärvoja kenttä kerrallaan JSON-tiedoston arvoihin.

KENTTÄKOHTAINEN TARKKUUS		
vendor_name		100.0% (99/99)
total_amount		100.0% (99/99)
receipt_date		100.0% (99/99)
receipt_type		100.0% (99/99)
is_deductible		100.0% (99/99)
vat_deductibility_percentage		100.0% (99/99)
vat_rates		100.0% (190/190)
vat_net_amounts		100.0% (190/190)
vat_amounts		100.0% (190/190)
vat_totals		98.9% (188/190)
vat_row_count		100.0% (99/99)
KOKONAISTARKKUUS		99.9%
VIRHEET KENTITTÄIN		
vat_totals (2 virhettä):		
x R-kioski Keskusasema (2025-03-09) VAT[10%]: GT=12.98 DB=13.98		
x ABC Liikennemyymälä (2024-06-29) VAT[25.5%]: GT=3.8 DB=4.8		

Toisessa testissä arvioitiin järjestelmän suorituskykyä käsin kirjoitetuilla kuiteilla, jotka ovat OCR-tunnistuksen kannalta huomattavasti haastavampia kuin painetut kuitit. Datasetsi sisälsi 30 käsin kirjoitettua kuittia, ja tarkkuudeksi mitattiin 96,8 %. Virheet keskittyivät VAT-summakenttiin, joissa käsialan tulkinta tuotti pieniä pyöristyspoikkeamia. Myyjän nimi, päivämäärä, kokonaissumma ja ALV-prosentti tunnistettiin kaikista kuiteista täydellisesti.

TARKKUUSRAPORTTI

Matchattu: 30/30

KENTTÄKOHTAINEN TARKKUUS

vendor_name		100.0% (30/30)
total_amount		100.0% (30/30)
receipt_date		100.0% (30/30)
receipt_type		100.0% (30/30)
is_deductible		100.0% (30/30)
vat_deductibility_percentage		100.0% (30/30)
vat_rates		100.0% (33/33)
vat_net_amounts		84.8% (28/33)
vat_amounts		90.9% (30/33)
vat_totals		90.9% (30/33)
vat_row_count		100.0% (30/30)

KOKONAISTARKKUUS

96.8%

VIRHEET KENTITTÄIN

vat_net_amounts (5 virhettä):

x Puutarha Kukka-Aino (2024-07-13) VAT[25.5%]: GT=26.73 DB=28.12

x Mäen Marja- ja Vihannestila (2026-07-16) VAT[14%]: GT=11.98 DB=1.68

x Mäen Marja- ja Vihannestila (2026-07-16) VAT[25.5%]: GT=6.13 DB=1.56

x Siivouspalvelu Puhdas Koti (2025-02-06) VAT[25.5%]: GT=68.99 DB=69.99

x Mökkimajoitus Järvenranta (2025-08-24) VAT[10%]: GT=156.55 DB=155.5

vat_amounts (3 virhettä):

x Mäen Marja- ja Vihannestila (2026-07-16) VAT[14%]: GT=1.68 DB=0.24

x Mäen Marja- ja Vihannestila (2026-07-16) VAT[25.5%]: GT=1.56 DB=0.4

x Mökkimajoitus Järvenranta (2025-08-24) VAT[10%]: GT=15.65 DB=15.55

vat_totals (3 virhettä):

x Mäen Marja- ja Vihannestila (2026-07-16) VAT[14%]: GT=13.66 DB=1.92

x Mäen Marja- ja Vihannestila (2026-07-16) VAT[25.5%]: GT=7.69 DB=1.96

x Mökkimajoitus Järvenranta (2025-08-24) VAT[10%]: GT=172.2 DB=171.05

Tulosten perusteella järjestelmä suoriutuu painettujen kuittien analysoinnista lähes virheettömästi. Käsin kirjoitettujen kuittien 96,8 % tarkkuus on merkittävä tulos ottaen huomioon, että käsialan OCR-tunnistus on yleisesti tunnettu haastava ongelma tietokoneen näkemisen alalla. On myös huomionarvoista, että kaikki havaitut virheet liittyivät yksittäisiin numeerisiin kenttiin. Järjestelmä ei sekoittanut kuitteja tai tuottanut täysin keksittyjä arvoja.

Ohjelmisto on suunnattu suomalaisille pienyrittäjille, joiden kuitit ovat tyypillisesti painettuja ja selkeärakenteisia, joten 99,9 % tarkkuus kuvastaa realistista käyttöympäristöä parhaiten.

8.1 Käyttäjätyytyväisyys

Koska varsinaista laajempaa käyttäjätestausta ei tämän tutkimuksen puitteissa ehditty toteuttaa, käyttäjätyytyväisyyttä arvioitiin suppeasti antamalla sovelluksen demoversio muutaman tuttavayrittäjän ja opiskelijan kokeiltavaksi. Testaajia pyydettiin lataamaan järjestelmään pari omaa kuittia, tarkastelemaan OCR:n tunnistamia tietoja, kokeilemaan raporttien muodostusta sekä kokeilemaan AI-assistentin chattia ja raporttien generointia. Testin jälkeen testaajilta kerättiin vapaamuotoinen sanallinen palaute.

Palaute oli pääosin myönteistä ja sovellusta keuhuttiin ammattimaiseksi. Erityisesti kuitin lataamisen ja AI:n tunnistamien tietojen tarkastamisen koettiin olevan intuitiivista ja nopeaa – yksikään testaaja ei tarvinnut erillistä opastusta päästäkseen alkuun. AI-assistentin koettiin tuovan lisäarvoa etenkin tilanteissa, joissa käyttäjä ei ollut varma, mihin kategoriaan tietty kulu kuuluisi. Kehitettävää nähtiin myös yksittäisen kuitin tarkastelunäkymässä (receipt/id). Nykyisessä toteutuksessa käyttäjä pääsee tarkastelemaan kuitin tarkempia tietoja ja alkuperäistä kuittikuvaa vain klikkaamalla kuitin auki yleisestä kuittilistauksesta, jolloin vertailu OCR:n tunnistamien tietojen ja alkuperäisen kuitin välillä onnistuu. Yksittäiseltä tarkastelusivulta ei kuitenkaan pääse suoraan seuraavaan tai edelliseen kuittiin, vaan käyttäjän on aina palattava takaisin yleisnäkymään ja valittava sieltä seuraava tarkasteltava kuitti erikseen. Testaajat kokivat tämän hidastavan useamman kuitin peräkkäistä läpikäyntiä, ja toivoivat mahdollisuutta selata kuitteja suoraan yksittäisen kuitin näkymästä esimerkiksi "seuraava" ja "edellinen" -painikkeiden avulla.

Testaajat myös korostivat, että olisi kiva, jos heti tiedostonlataus vaiheessa, jossa AI ja OCR analysoi kuitin, näkyisi analysoitu data kuitista eikä vasta "receipts" sivulla. Omasta mielestä tämä on hieman vaikea tilanne sillä upload sivun täytyy olla suhteellisen selkeä ja minimaalinen ja analysoitu data tekisi siitä ehkä hieman liian raskaan näköisen.

On kuitenkin syytä korostaa, että kyseessä oli hyvin pieni ja epämuodollinen otos, eikä tulosta voida pitää tilastollisesti yleistettävänä tai tieteellisesti pätevänä käytettävyystudkimuksena. Otos ei myöskään edustanut kattavasti eri-ikäisiä tai eri toimialoilla toimivia pienyrittäjiä, eikä testissä käytetty vakiintunutta käytettävyysskyselyä (esim. SUS-mittaria). Luotettavan kuvan saamiseksi sovelluksen todellisesta käyttäjätyytyväisyydestä vaadittaisiin jatkossa laajempi, useamman kymmenen käyttäjän otoksella ja standardoidulla mittaristolla toteutettu käytettävyystudkimus todellisessa käyttöympäristössä.

9 Pohdinta

Tämän tutkimuksen tavoitteena oli suunnitella ja toteuttaa tekoälypohjainen kirjanpitosovellus, joka vähentää pienyrittäjien manuaalisen kirjanpityksen määrää, sekä arvioida, missä määrin tekoäly soveltuu tähän tarkoitukseen. Design Science Research -viitekehys osoittautui tarkoitukseensa hyvin sopivaksi, sillä se ohjasi työtä systemaattisesti ongelman tunnistamisesta aina toimivan artefaktin arviointiin asti, ja Agile-kehitysmalli tuki tätä sykliä käytännön tasolla mahdollistamalla vaatimusten tarkentumisen kehityksen edetessä.

Pienten yritysten keskeisimmät kirjanpidon haasteet liittyvät kuittien ja tositteiden keräämiseen, manuaaliseen tietojen syöttämiseen sekä ALV-laskelmien oikeellisuuden varmistamiseen. Näistä automaatio tuo eniten hyötyä juuri tietojen syöttämiseen: OCR ja tekoäly poistavat tarpeen kirjata myyjä, summa, päivämäärä ja ALV-tiedot käsin jokaiselta kuitilta erikseen. Lisäksi automaattinen ALV-raportointi vähentää laskentavirheitä ja helpottaa arvonlisäveroilmoitusten sekä veroilmoituksen tekemistä. Yhdessä nämä ominaisuudet vastaavat suoraan niihin työvaiheisiin, jotka vievät pienyrittäjältä eniten aikaa suhteessa niiden tuottamaan arvoon.

Ajansäästö ohjelmistolla oli erittäin merkittävä niin kuin osasi odottaakin. 100 kuitin käsittely vei ainoastaan 7 minuuttia aikaa ja tekoäly analysoi kuitit lähes 100 % tarkkuudella. Toki voi heittää ainoastaan arvioita paljonko keskiverto yrittäjällä menisi aikaa laittaa 100 kuittia kirjanpitoon, mutta keskiarvoksi oman ja muiden läheisten yrittäjien kokemusten perusteella kuitin

laittaminen esimerkiksi Excel taulukkoon vie aikaa keskimäärin noin 20–60 sekuntia, mikä kattaa kuittien lajittelun, tietojen syöttämisen (myyjän nimi, päivämäärä, summa, ALV) sekä inhimilliset virheet. Eli 100 kuitin kohdalla noin 30–100 minuuttia.

Tutkimuksen ideana oli myös selvittää, kuinka tarkka toteutettu prototyyppi olisi. Tähänkin saatiin erinomainen vastaus testidatalla, mikä toteutettiin kappaleessa 8. 99.9 % tarkkuus painetuilla kuiteilla sekä 96.8 % tarkkuus käsin kirjoitetuilla kuiteilla on todella hyvä ja ylitti odotukseni suuresti. Kuitenkin, vaikka 99.9 % tarkkuus on erinomainen, täydelliseen automaatioon ei tule luottaa vaan yrittäjän on hyvä silmäillä kuitit läpi ja etenkin tapauksissa, joissa syötetty kuitti on huono kuntoinen eikä arvot ole helposti erotettavissa.

Työhön kuitenkin liittyi useita rajoituksia, jotka on syytä tuoda esiin avoimesti. Ensinnäkin kyseessä oli yhden kehittäjän toteuttama prototyyppi, mikä rajasi sekä toteutuksen laajuutta että testauksen kattavuutta. Erityisesti käytettävyytestaus ja käyttäjätyytyväisyyden mittaaminen jäivät heikolle tasolle eikä niitä pystytty toteuttamaan kymmenillä käyttäjillä. Toiseksi tietosuojan ja lainsäädäntöön liittyvät kysymykset erityisesti OpenAI:n rajapinnan kautta tapahtuva datan siirto EU:n ulkopuolelle jäivät tunnistetuiksi, mutta ei täysin ratkaistuiksi. Tuotantokäyttöön siirryttäessä tietosuojaseloste, käyttöehdot ja tiedonsiirtomekanismit vaatisivat lakimiehen tarkistuksen.

10 Lähteet

Suomen yrittäjät, "Yrittäjyys Suomessa", <https://www.yrittajat.fi/tietoa-meista/tietoa-yrittajarjestosta/yrittajyys-suomessa/>, Helsinki, 2026.

Pasuksmit. J, "Investigating documented information for accurate effort estimation in agile software development", ESEC/FSE 2021: Proceedings of the 29th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, University of Melbourne, Australia, pp. 1605–1609.

KT Counter Oy, "Kirjanpidon hinta 2026: Mitä pienyrityksen kirjanpito maksaa?", <https://www.ktcounter.fi/kirjanpito/kirjanpidon-hinta/>, Joensuu, 2026.

Dilmegani. C, Alper. S, "OCR Benchmark: Text Extraction / Capture Accuracy", <https://aimultiple.com/ocr-accuracy>, 2026.

Abbadia. J, "Mitä eroa on: Kvalitatiivinen vs. kvantitatiivinen tutkimus?", <https://mindthegraph.com/blog/fi/kvalitatiivinen-vs-kvantitatiivinen-tutkimus/>, 2023.

Siemon. D, "Design Science Research", https://www.softwareengineering.fi/wp-content/uploads/2024/11/FAST_introduction_to_DSR.pdf, LUT University.

Kamisetty. A, Narsina D, Rodriquez. M, Kothapalli. S, "Microservices vs. Monoliths: Comparative Analysis for Scalable Software Architecture Design", Engineering International, Volume 11, No. 2 (2023), USA, 2023

Hura. G. S, "Client-server computing architecture: an efficient paradigm for project management", Proceedings for Operating Research and the Management Sciences, Singapore, 1995, pp. 146-152

Gupta, L. (2025). What Is REST? What is REST?: REST API Tutorial. (24.11.2025)

Next.js Documentation, <https://nextjs.org/>, 2024

jwt-documentation, <https://www.jwt.io/introduction#difference-decoding-encoding-jwt>, 2026

Cloudflare, <https://www.cloudflare.com/learning/ssl/what-is-https/>, 2026

Euroopan Unioni, https://europa.eu/youreurope/business/governance-and-sustainability/digital-and-data-compliance/data-protection-gdpr/index_fi.htm, 12.12.2025

Peffers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A Design Science Research Methodology for Information Systems Research. *Journal of Management Information Systems*, 24(3), 45–77.